Sistemi Operativi

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Sistemi Operativi" erogato nell'A.A. 2026 dal professore Gabriele Tolomei per il C.d.L Informatica (33503)*

13 ottobre 2026

Indice

1 Introduzione all'hardware dei calcolatori 4

1.1 Lo spettro Hardware-Software 5

1.2 L'architettura di von Neumann (1945) 6

1.2.1 Il Bus di Sistema 8

1.2.2 Il ciclo di istruzione 8

1.2.3 L'Unità Centrale di Elaborazione (CPU) 9

1.2.4 Registri della CPU 10

1.3 Pipelining delle istruzioni 10

1.3.1 Hazard della pipeline 11

1.4 Instruction Set Architectures (ISA) 12

1.5 Processori Multi-Core 12

1.5.1 Prestazioni della CPU: Velocità di Clock e Cicli 12

1.6 La gerarchia di memoria 14

1.6.1 Il principio di località 14

1.6.2 Architettura della Cache Memory 15

1.6.3 Politiche di scrittura della Cache 16

1.6.4 Politiche di mapping della Cache Line 16

1.7 Il sistema Input/Output (I/O) 17

1.7.1 Protocolli di trasferimento I/O 18

2 Introduzione ai Sistemi Operativi 19

2.1 Cos'è un sistema operativo? 19

2.2 I livelli di astrazione forniti dal SO 19

2.3 Processi e Thread 20

2.4 La virtualizzazione della memoria 21

2.5 Perché il Sistema Operativo necessita di supporto hardware 21

2.5.1 Esecuzione in due modalità 21

2.6 Protezione degli accessi in memoria 24
2.6.1 Memory Management Unit (MMU) 25

3 Il linguaggio di programmazione C 26
3.1 Le variabili in C 26
3.1.1 Visibilità delle variabili 27
3.1.2 I tipi di variabili 28
3.2 Gli Array 28
3.3 Le stringhe 29
3.3.1 Funzioni per le stringhe 30
3.4 I puntatori 30
3.4.1 Aritmetica dei puntatori 31
3.5 La gestione della memoria 32
3.5.1 Lo Stack 32
3.5.2 Lo Heap 33
3.5.3 Errori di memoria 34
3.6 La traduzione da sorgente ad eseguibile 35
3.6.1 Il preprocessore 35
3.6.2 La compilazione 36
3.6.3 L'assemblaggio 36
3.6.4 Il linking 36
3.6.5 Librerie statiche e dinamiche 37

4 Processi e API di sistema 38
4.1 L'astrazione dei processi 38
4.2 Gli stati di un processo 39
4.3 Il Process Control Block (PCB) 40
4.4 La gerarchia dei processi 40
4.5 La funzione fork() 41
4.5.1 La filosofia Copy-on-Write 44

4.6 La sincronizzazione dei processi con wait() 45
4.7 La funzione exec() 46
4.8 Altre API di sistema 48

5 Scheduling della CPU 49

5.1 Assunzioni sul carico di lavoro e metriche 49
5.2 I primi passi: FIFO e SJF 51
5.2.1 FIFO - First In, First Out 51
5.2.2 SJF - Shortest Job First 52
5.3 Interruzione dei processi e STCF 54
5.4 Round Robin (RR) 55
5.5 Incorporazione dell'I/O 57

1

Introduzione all'hardware dei calcolatori

Per comprendere appieno il ruolo dei sistemi operativi, è utile richiamare una celebre affermazione del matematico e informatico Richard Hamming:

> *«Lo scopo del calcolo è la comprensione, non i numeri.»* — Richard Hamming

In modo del tutto analogo, possiamo affermare che lo scopo primario di un **Sistema Operativo** è l'**astrazione**, e non la semplice gestione meccanica dell'hardware.

Tuttavia, per poter costruire astrazioni corrette, stabili ed efficaci, è indispensabile padroneggiare in primo luogo il funzionamento della macchina fisica sottostante.

Osservazione 1.0.1: Virtualizzazione dell'hardware

I buoni sistemi operativi si distinguono proprio per la loro capacità di **virtualizzare** con successo i diversi aspetti dell'architettura hardware, fornendo ai programmi applicativi un'interfaccia semplificata e indipendente dai dettagli implementativi del livello fisico.

In questo capitolo introduttivo verranno dunque fornite delle basi riguardanti l'architettura degli elaboratori, in quanto queste conoscenze sono essenziali e necessarie al fine di comprendere *come* e *dove* lavora il Sistema Operativo. In particolare verranno trattate:

1. **L'architettura di von Neumann**: la struttura e il layout generale dell'elaboratore.
2. **La CPU**: l'organizzazione interna comprendente registri, ALU, pipeline e suddivisione in core.

3. **La gerarchia di memoria**: le tecnologie SRAM/DRAM, il principio di località e la gestione delle memorie cache.

4. **Il sistema di Input/Output (I/O)**: il ruolo dei controller, la gestione degli interrupt e il meccanismo DMA (*Direct Memory Access*).

1.1 Lo spettro Hardware-Software

Per comprendere il ruolo di un sistema operativo, è fondamentale analizzare la sua collocazione all'interno della gerarchia che separa il software applicativo dall'hardware sottostante. Ci si chiede quindi: *dove si posiziona esattamente il sistema operativo?*

L'architettura del sistema può essere suddivisa in tre macro-aree di esecuzione: lo **User Space** (spazio utente), il **Kernel Space** (spazio kernel) e la **Physical Machine** (macchina fisica).

Livello dell'Architettura	**Ambito di Esecuzione**
Applicazioni (*Python, Java, C++, Web*)	**User Space**
Librerie di Sistema (*libc*)	**User Space**
Sistema Operativo (*Kernel*)	**Kernel Space**
Firmware / Device Driver	**Hardware / Low-Level Space**
Hardware (*CPU, RAM, Controller I/O*)	**Physical Machine**

Parleremo in modo più approfondito del kernel e delle componenti di un Sistema Operativo più avanti nel corso. Come possiamo però vedere, il sistema operativo agisce come un vero e proprio *ponte* tra il livello logico (le applicazioni in esecuzione) e il livello fisico (gli effettivi componenti hardware).

Le astrazioni software fornite dai sistemi operativi devono necessariamente rispettare i limiti fisici imposti dall'hardware sottostante. La progettazione del software di sistema non può prescindere da questi vincoli, tra i quali spiccano in particolare:

Divario di velocità (*Speed gap*): Le CPU operano a frequenze molto elevate, riuscendo ad eseguire le istruzioni in frazioni di nanosecondo. Al contrario, i tempi di accesso alla memoria principale (DRAM) si misurano nell'ordine delle decine di nanosecondi. Questa marcata differenza prestazionale rende la memoria un frequente collo di bottiglia durante l'esecuzione dei processi.

Compromesso sulla memorizzazione (*Storage trade-off*): Esiste un vincolo fisico ed economico inscindibile tra velocità, capacità e costo della memoria. I dispositivi di storage più veloci (come le cache) sono estremamente costosi e di piccole dimensioni, mentre le memorie più economiche (come i dischi secondari) garantiscono capacità enormi ma a fronte di velocità decisamente inferiori.

Concorrenza (*Concurrency*): L'esecuzione parallela nei reali sistemi multi-core richiede un supporto hardware esplicito e dedicato per la sincronizzazione (ad esempio tramite istruzioni atomiche), indispensabile per evitare race condition e garantire la coerenza dei dati.

Osservazione 1.1.1: Ruolo del Sistema Operativo

Uno degli scopi primari nella progettazione degli algoritmi di un sistema operativo è proprio quello di mascherare ed erodere questi limiti fisici, offrendo ai programmi un'astrazione dell'hardware efficiente, omogenea e trasparente.

1.2 L'architettura di von Neumann (1945)

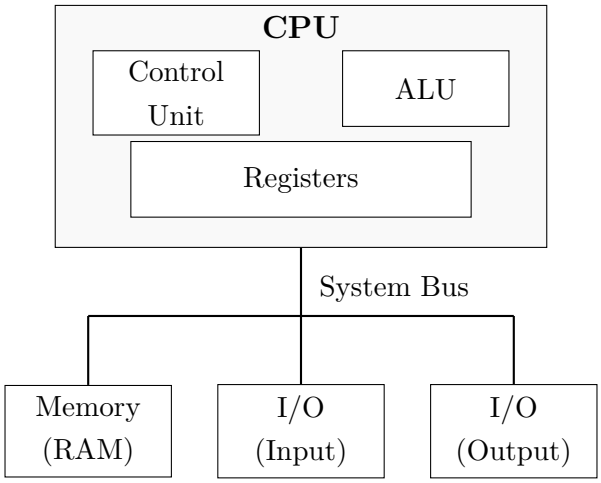
L'architettura di von Neumann rappresenta il fondamento teorico di quasi tutti i moderni calcolatori d'uso generale (*general-purpose*).

Tale modello si basa su due concetti chiave:

Concetto di programma memorizzato (*Stored-Program*): Le istruzioni e i dati condividono lo stesso spazio di memoria fisica (memorizzati in una memoria detta *Memoria Centrale*).

Esecuzione sequenziale: Le istruzioni vengono prelevate ed estratte una alla volta dalla memoria ed eseguite sequenzialmente dalla CPU. Non è previsto alcun tipo di concorrenza da questa architettura.

Di seguito forniamo uno schema grafico rappresentante la struttura del modello:



L'architettura si articola nei seguenti elementi funzionali:

Unità Centrale di Elaborazione (CPU): È il «cervello» del calcolatore, responsabile del coordinamento e dell'esecuzione di tutte le istruzioni. Al suo interno comprende:

- **Control Unit (Unità di Controllo):** Preleva le istruzioni dalla memoria, le decodifica e genera i segnali di controllo necessari al loro completamento.
- **ALU (Arithmetic Logic Unit):** Esegue le operazioni aritmetiche di base e le operazioni logiche elementari sui dati.
- **Registers (Registri):** Celle di memoria ad altissima velocità situate all'interno della CPU, impiegate per contenere temporaneamente dati, indirizzi di memoria o stati di esecuzione. Tra questi registri abbiamo, ad esempio:
  - **Program Counter (PC)**, che si occupa di memorizzare l'indirizzo della prossima istruzione che deve essere eseguita dalla CPU.
  - **Instruction Register (IR)**, che contiene il codice binario dell'effettiva istruzione puntata dal Program Counter.

Bus di Sistema (*System Bus*): È la struttura di collegamento condivisa (costituita da linee elettriche) attraverso la quale la CPU scambia indirizzi, dati e segnali di controllo con la memoria e le periferiche.

Memoria Principale (RAM): Contiene sia il codice dei programmi in esecuzione sia i dati necessari al funzionamento delle applicazioni. Nel modello di von Neumann, istruzioni e dati condividono lo stesso spazio di memoria ed i medesimi bus.

Interfacce di I/O (Input/Output): Moduli necessari a permettere l'interazione tra l'elaboratore e il mondo esterno, distinti in dispositivi di ingresso (*Input*, come tastiera e mouse) e di uscita (*Output*, come monitor e stampanti).

Tuttavia, come visto in precedenza, il principale svantaggio dell'architettura di von Neumann è proprio il fatto che le istruzioni e i dati condividono lo stesso spazio di memoria. Da ciò si origina il cosiddetto *von Neumann Bottleneck*, cioè un effettivo limite fisico in quanto una memoria non può essere acceduta da più dispositivi contemporaneamente. Un'alternativa a questo modello è infatti il cosiddetto **Modello Harvard**, che prevede spazi di memoria e bus fisicamente separati per istruzioni e dati.

1.2.1 Il Bus di Sistema

Entriamo più nel dettaglio riguardo ai bus di sistema, che hanno l'importante compito di traspostare qualsiasi tipo di dato tra una componente e l'altra.

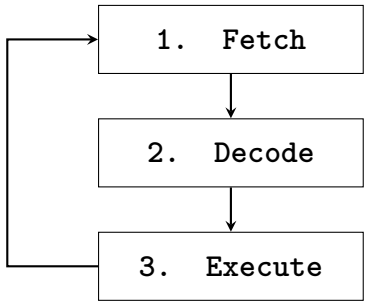
Bus Indirizzi (*Address Bus*): trasporta gli indirizzi di memoria dalla CPU verso i componenti di destinazione. Questo bus è **unidirezionale**, in quanto solo la CPU deve poter inviare l'indirizzo di memoria su cui lavorare. Inoltre, l'Address Bus ha una proprietà detta **Bus Width** (che fisicamente rappresenta quanti fili elettrici ci sono per quel bus) e che determina qual è il massimo numero di locazioni indirizzabili. La formula è molto semplice: se abbiamo width $= N$, il numero di locazioni indirizzabili è pari a 2^N.

Bus Dati (*Data Bus*): gestisce il trasferimento dei byte di dati effettivi tra i vari componenti. Rappresenta il canale sul quale viaggia fisicamente l'istruzione da eseguire oppure il dato da elaborare. Poiché i dati devono poter essere scritti e letti, la trasmissione lungo il Data Bus è di tipo *bidirezionale*. Anche questo bus possiede la proprietà di **Bus Width**, con significato analogo a quella dell'Address Bus, e questo valore in genere coincide con la Word Size del processore (dunque più è ampio il bus, maggiore è la quantità di dati che possono essere trasmessi simultaneamente).

Bus di Controllo (*Control Bus*): veicola i segnali di comando e di sincronizzazione del sistema, come le operazioni di lettura (*read*), scrittura (*write*), segnali di *reset* o richieste di *interrupt*. In sostanza, è il mezzo tramite cui la Control Unit (CU) definisce cosa fare con il dato e l'indirizzo selezionati.

1.2.2 Il ciclo di istruzione

Il **ciclo di istruzione** (o *fetch-decode-execute cycle*) rappresenta il processo fondamentale alla base del funzionamento di qualsiasi elaboratore. Ogni programma esegue questo ciclo in maniera continua.



Il ciclo si compone di tre fasi sequenziali gestite direttamente dalla CPU:

1. **Fetch (Prelevamento):** il contenuto del PC viene trasferito sull'Address Bus per poi prelevare l'effettiva istruzione puntata da quell'indirizzo. Alla fine di questa fase il valore del PC viene incrementato in modo di far passare all'istruzione successiva.
2. **Decode (Decodifica):** il processore analizza e interpreta il significato della sequenza di bit che compone l'istruzione. Questa operazione viene svolta in base alla specifica codifica scelta dall'architettura dell'elaboratore (ossia l'*Instruction Set Architecture* o ISA).
3. **Execute (Esecuzione):** viene svolta l'operazione richiesta dall'istruzione. Questa fase può comportare un calcolo eseguito dall'Unità Aritmetico-Logica (ALU) oppure un'operazione di lettura/scrittura in memoria.

Al termine della fase di esecuzione, il ciclo riparte immediatamente dalla fase di *fetch* per l'istruzione successiva.

1.2.3 L'Unità Centrale di Elaborazione (CPU)

La **Central Processing Unit** (**CPU**) rappresenta il "cervello" del calcolatore, essendo l'organo fondamentale preposto all'esecuzione delle istruzioni memorizzate in memoria.

L'architettura interna della CPU si articola su tre elementi costruttivi principali:

1. **Datapath (Unità Dati):** include l'**Arithmetic Logic Unit** (**ALU**), preposti alla manipolazione e al calcolo effettivo sui dati e i registri General-Purpose (adibiti al contenimento di risultati intermedi per le operazioni della ALU).
2. **Control Unit** (**CU**): opera come coordinatore dell'intero microprocessore. Il suo compito è prelevare le istruzioni dalla memoria (*fetch*), decodificarle e pilotare opportunamente il datapath per consentirne l'esecuzione.
3. **Bus Interno:** insieme di canali e linee di interconnessione interne che mettono in comunicazione diretta i registri, l'ALU e la Control Unit.

1.2.4 Registri della CPU

Vediamo ora una carrellata veloce dei registri di sistema e delle loro funzionalità:

1. **Registri visibili all'utente (General-Purpose):** sono direttamente indirizzabili dalle istruzioni in linguaggio assembly per la manipolazione dei dati e l'esecuzione delle operazioni.
2. **Registri di controllo e di stato (Special-Purpose):** sono utilizzati dalla *Control Unit* (CU) per gestire lo stato di esecuzione del sistema e del processore. Tra i più rilevanti rientrano:
   - **Program Counter (PC):** mantiene l'indirizzo della prossima istruzione da eseguire.
   - **Instruction Register (IR):** contiene l'istruzione attualmente in fase di decodifica o esecuzione.
   - **Processor Status Register (PSR):** memorizza le informazioni sullo stato corrente del processore (flag di condizione, livello di privilegio, ecc.).
   - **Stack Pointer (SP):** contiene l'indirizzo di memoria della cima dello stack delle chiamate (ad esempio di chiamate delle funzioni).
   - **Memory Address Register (MAR):** contiene l'indirizzo che sta per essere utilizzato dalla RAM. E' direttamente collegato all'Address Bus.
   - **Memory Data Register (MDR):** contiene i byte di dati letti dalla RAM, ed è direttamente collegato al Data Bus.

1.3 Pipelining delle istruzioni

Nei processori più semplici, il ciclo di esecuzione delle istruzioni segue un approccio strettamente sequenziale. Come previsto dal modello classico dell'architettura di Von Neumann, il processo avviene in modo sequenziale: la CPU deve completare interamente le fasi di prelievo (*fetch*), decodifica (*decode*) ed esecuzione (*execute*) di una singola istruzione prima di poter passare a quella successiva. Questo schema, pur essendo semplice, comporta un notevole spreco di risorse hardware, poiché molte unità funzionali della CPU rimangono inattive durante le fasi in cui non sono direttamente coinvolte.

Per superare questa limitazione e migliorare le prestazioni del sistema, si introduce una tecnica di ottimizzazione fondamentale:

Definizione 1.3.1: Pipelining

Il **pipelining** è una tecnica di ottimizzazione hardware che consente di sovrapporre l'esecuzione di più istruzioni nel tempo, parallelizzando le diverse fasi del loro ciclo di vita.

Tale tecnica, almeno per le architetture di tipo RISC, si articola in 5 stadi:

1. **IF (Instruction Fetch):** prelievo dell'istruzione dalla cache di primo livello (L1 Cache) o dalla memoria principale (RAM).
2. **ID (Instruction Decode):** decodifica dell'istruzione appena prelevata e lettura dei registri sorgente.
3. **EX (Execute):** esecuzione del calcolo tramite l'unità aritmetico-logica (ALU) o calcolo dell'indirizzo di memoria.
4. **MEM (Memory Access):** accesso alla memoria dati, che comporta un'operazione di lettura o scrittura verso la cache dati.
5. **WB (Writeback):** scrittura del risultato finale all'interno del registro di destinazione.

Grazie a questa suddivisione, ogni stadio opera su un'istruzione differente in modo simultaneo durante ciascun ciclo di clock, consentendo un'elevata sovrapposizione delle operazioni e migliorando l'efficienza del processore.

1.3.1 Hazard della pipeline

In condizioni ideali, le architetture con pipeline consentono di raggiungere il massimo incremento prestazionale. Tuttavia, nella realtà possono verificarsi delle situazioni anomale note come **hazard** (o criticità), che impediscono all'istruzione successiva di essere eseguita nel ciclo di clock prestabilito.

Tali criticità si dividono principalmente in tre grandi categorie:

Hazard strutturali (*Structural Hazards*): Si verificano in presenza di un conflitto di risorse, ovvero quando due o più stadi della pipeline richiedono contemporaneamente l'accesso allo stesso componente hardware fisico.

Hazard sui dati (*Data Hazards*): Si manifestano quando un'istruzione dipende strettamente dal risultato di un'istruzione precedente che non ha ancora completato il suo percorso all'interno della pipeline.

Hazard di controllo (*Control Hazards*): Sono causati dalle istruzioni di salto condizionato o incondizionato (*branch*), le quali modificano il valore del *Program Counter* (PC). Ciò rende invalide le istruzioni che sono già state caricate (*fetched*) preventivamente nella pipeline.

Per risolvere tali criticità, le CPU moderne impiegano dei predittori di salto (**branch predictors**) per indovinare l'esito di un'istruzione di salto prima che essa venga effettivamente completata. Tuttavia, tali metodologie non verranno trattate.

1.4 Instruction Set Architectures (ISA)

Nel mondo dell'informatica sono stabiliti due standard principali riguardo al linguaggio adottato dalle istruzioni macchina:

CISC (Complex Instruction Set Computer): ogni istruzione macchina può comprenderne altre (sono infatti a lunghezza variabile), in modo da "semplificare" la scrittura del software per gli utenti.

RISC (Reduced Instruction Set Computer): mantiene solo le istruzioni principali del codice macchina (tutte hanno la stessa lunghezza).

1.5 Processori Multi-Core

Gli elaboratori di oggi aggirano i limiti di potenza e velocità dei processori collocando più processori (detti appunto *core*) su un singolo macro-processore. La caratteristica principale è proprio che ogni *core* è come se fosse "autonomo", in quanto possiede i suoi registri e le sue unità di controllo. A questo punto deve essere il Sistema Operativo a saper gestire l'architettura Multi-Core: deve saper far comunicare efficientemente i vari core per la condivisione di risorse senza che si verifichino problemi di sincronizzazione delle variabili. Discuteremo di queste tecniche quando parleremo di *concorrenza* e *multi-threading*.

1.5.1 Prestazioni della CPU: Velocità di Clock e Cicli

Il funzionamento della CPU è sincronizzato da un **clock di sistema** (*SystemClock*) oscillante, che scandisce il ritmo delle operazioni eseguite dal processore. Per comprendere le prestazioni di una CPU, è fondamentale analizzare due grandezze fondamentali legate a questo segnale:

- **Periodo di clock (T):** rappresenta la durata temporale di un singolo ciclo di clock (ad esempio 0.25 nanosecondi).

- **Frequenza di clock (f):** rappresenta il numero di cicli di clock al secondo, misurata in Hertz (Hz).

Queste due grandezze sono inversamente proporzionali tra loro e legate dalla relazione:

$$f = \frac{1}{T}$$

A titolo d'esempio, una CPU a 4.0 GHz è in grado di completare 4×10^9 cicli di clock in un solo secondo.

Tuttavia, non tutte le istruzioni macchina richiedono lo stesso tempo per essere eseguite: le istruzioni più semplici possono essere completate in un solo ciclo di clock, mentre le operazioni più complesse (come la divisione tra numeri in virgola mobile o gli accessi alla memoria principale) richiedono un numero elevato di cicli per poter essere portate a termine.

Per valutare e confrontare l'efficienza dei processori, risulta fondamentale definire delle metriche standard. Una delle domande chiave nell'architettura dei calcolatori è: quante istruzioni riesce ad eseguire una CPU per ogni ciclo di clock? Per rispondere a questo interrogativo, introduciamo due parametri fondamentali:

- **CPI (Cycles Per Instruction):** rappresenta il numero medio di cicli di clock necessari per eseguire una singola istruzione.
- **IPC (Instructions Per Cycle):** rappresenta l'inverso del CPI, ovvero il numero di istruzioni eseguite in un singolo ciclo di clock. La relazione matematica è la seguente:

$$\text{IPC} = \frac{1}{\text{CPI}}$$

Per calcolare il tempo totale necessario per eseguire un determinato programma, si utilizza la formula fondamentale delle prestazioni della CPU, che mette in relazione il numero di istruzioni, l'efficienza architetturale e la frequenza di clock:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Dove con *CPU Time* si intende il tempo totale necessario per eseguire tutte le istruzioni del programma. Analizzando i singoli componenti della formula:

- **Instruction Count:** il numero totale di istruzioni eseguite dal programma.
- **CPI:** il numero medio di cicli per istruzione.
- **Clock Cycle Time:** la durata di un singolo ciclo di clock (l'inverso della frequenza di clock).

1.6 La gerarchia di memoria

La velocità dei processori, nel corso degli ultimi anni, è cresciuta molto più velocemente (si potrebbe dire in maniera quasi esponenziale) rispetto alla velocità di accesso alle memorie RAM. Per continuare a velocizzare l'accesso alle risorse, gli architetti dei computer fanno riferimento ad una *gerarchia delle memorie*, una struttura piramidale che organizza le memorie dall'alto verso il basso, dove quella più in **cima** sono i **registri**, memorie *estremamente veloci*, situate all'interno della CPU, ma altrettanto costose; alla base di questa piramide si collocano i dispositivi di archiviazione di massa, come *SSD* e *HDD*, che sono più economiche ma decisamente più lente. Dunque, il compito di un bravo architetto di computer è quello di trovare il giusto compromesso a livello di prezzo e velocità, e il compito di un buon sistema operativo è quello di saper sfruttare queste risorse nel modo più efficiente possibile.

Possiamo riassumere il comportamento della struttura con una regola fondamentale: **scendendo nella gerarchia, la velocità di accesso diminuisce, la capacità aumenta e il costo per byte diminuisce**.

Livello	Componente	Capacità tipica	Latenza di accesso
L0	CPU Registers	< 1 KiB	< 0.5 ns
L1	L1 Cache (per core)	$32 - 64$ KiB	$1 - 2$ ns
L2	L2 Cache (per core)	$256 - 512$ KiB	$3 - 5$ ns
L3	L3 Cache (shared)	$8 - 64$ MiB	$10 - 20$ ns
L4	Main Memory (DRAM)	$8 - 64$ GiB	$60 - 100$ ns
L5	SSD (NAND Flash)	256 GiB - 2 TiB	$10 - 100$ μs
L6	HDD (Magnetic Disk)	$1 - 10$ TiB	$5 - 10$ ms

La memoria principale è strutturata come una griglia monodimensionale di indirizzi di byte (*byte addresses*). Ogni cella possiede un indirizzo univoco a cui corrisponde un determinato contenuto memorizzato.

1.6.1 Il principio di località

L'efficacia della gerarchia di memoria (*memory hierarchy*) si basa sul "principio di località", che definisce una serie di principi secondo i quali lavora la CPU per ottimizzare l'accesso

alle informazioni. I programmi in esecuzione non accedono infatti alla memoria in modo casuale, ma presentano due forme fondamentali di località:

Località temporale (*Temporal Locality*): se una determinata locazione di memoria viene acceduta una volta, è molto probabile che la stessa locazione venga acceduta nuovamente nel futuro prossimo. Ad esempio, in un ciclo `for` in cui si accede alla variabile `i` per ogni iterazione del ciclo, la CPU dovrà accedere sempre alle stesse variabili e dunque terrà memorizzato in qualche registro vicino alla CPU l'indirizzo di memoria di quelle variabili.

Località spaziale (*Spatial Locality*): se viene acceduta una specifica locazione di memoria, è molto probabile che anche le locazioni ad essa vicine o adiacenti vengano accedute nel prossimo futuro. Tenendo sempre l'esempio del ciclo `for`, vedendo che tali locazioni di memoria vengono usate spesso, la CPU tende a memorizzare quei valori nei registri più vicini ad essa, in modo da renderne l'accesso quasi immediato.

1.6.2 Architettura della Cache Memory

Le memorie cache non gestiscono i dati a livello di singoli byte, ma operano su blocchi di dimensione fissa denominati **cache line** (la cui dimensione tipica è di 64 byte).

Ogni entry all'interno di una cache line è composta da tre elementi fondamentali:

Valid Bit: Un bit di controllo che indica se i dati presenti nella linea di cache sono validi ed aggiornati. Questo flag permette di verificare se il dato cercato è effettivamente presente ed utilizzabile all'interno della cache.

Tag Bit: Identifica l'indirizzo di memoria originale (in RAM) da cui sono stati reperiti i dati. Agisce in modo analogo alla chiave di un dizionario: permette di stabilire in modo univoco a quale posizione della memoria principale appartenga il blocco.

Data Block: Contiene l'effettivo blocco di dati copiato dalla memoria RAM (tipicamente 64 byte), ovvero il contenuto informativo richiesto situato a quell'indirizzo.

Nel momento in cui la CPU richiede un determinato indirizzo di memoria (*memory address*), si possono verificare due situazioni distinte:

Cache Hit (Dato trovato) Si ha un *cache hit* quando la CPU individua con successo l'indirizzo e il dato cercato direttamente all'interno della memoria cache.

Cache Miss (Dato non trovato) Si ha un *cache miss* quando la CPU non trova l'indirizzo richiesto all'interno della memoria cache, rendendo necessario il recupero del dato da memorie più lente.

1.6.3 Politiche di scrittura della Cache

Nel momento in cui la CPU scrive i dati in un cache block, possono essere applicate due politiche per mantenere aggiornata la RAM:

Write-Through: la CPU scrive contemporaneamente sia nella cache che nella memoria centrale. Il vantaggio è ovviamente una sincronizzazione costante dei dati; lo svantaggio è che una semplice operazione di scrittura di un dato sarà sempre lenta in quanto si dovrà riportare anche in memoria centrale, e dunque elimina il vantaggio di utilizzo della cache.

Write-Back: la CPU scrive solo nella cache line e la marca come "modificata". L'aggiornamento effettivo nella memoria centrale si verifica solo quando tale cache line viene espulsa dalla cache per fare spazio ad altri dati. Vantaggioso in quanto viene mantenuta la velocità di memorizzazione di dati all'interno della cache, ma rimane il problema possibile della sincronizzazione.

1.6.4 Politiche di mapping della Cache Line

Una questione fondamentale nell'organizzazione delle architetture dei calcolatori riguarda il criterio secondo cui un blocco della memoria principale (*memory block*) viene posizionato all'interno di uno slot della memoria cache (*cache slot*).

Per gestire questa mappatura ed individuare la posizione specifica in cui collocare i dati, si adottano principalmente tre schemi di progettazione:

Direct Mapped (Mappatura Diretta): Ogni blocco di memoria principale viene mappato in modo univoco ed esatto in un solo ed unico *cache slot*. Vantaggioso in quanto siamo sicuri di dove ritrovare i blocchi di memoria della cache. Possono tuttavia verificarsi conflitti di indirizzi in caso di mappatura nella stessa locazione.

Fully Associative (Completamente Associativa): Ogni blocco di memoria principale può essere collocato in un qualsiasi *cache slot* libero all'interno della cache. Tale approccio elimina i problemi di conflitto di mappatura, ma potendo posizionare un blocco in qualsiasi punto, dovremo cercare in qualsiasi punto possibile.

Set Associative (Associativa a Insiemi): Rappresenta un compromesso tra le due politiche precedenti. La memoria cache viene suddivisa in un determinato numero di insiemi (*set*), ciascuno composto da un numero fisso di slot. Un blocco di memoria viene mappato in modo rigido a uno specifico *set*, ma all'interno di quel set può risiedere in un qualsiasi slot libero.

1.7 Il sistema Input/Output (I/O)

Affinché un calcolatore sia effettivamente utile ed utilizzabile, esso deve necessariamente comunicare con l'ambiente esterno mediante opportuni dispositivi. È importante sottolineare che i dispositivi di Input/Output non si limitano unicamente alle periferiche di interazione diretta con l'utente (come quelle adibite all'inserimento di testo o alla visualizzazione grafica), ma comprendono un'ampia varietà di categorie:

- **Dispositivi di memorizzazione secondaria (Storage Devices):** memorie di massa persistenti quali SSD, HDD e unità USB Flash.
- **Dispositivi di rete (Network Devices):** schede di rete (NIC), sia cablate (Ethernet) che wireless (Wi-Fi), per la trasmissione e ricezione di dati attraverso la rete.
- **Interfacce utente (User Interfaces):** periferiche standard di interazione uomo-macchina, tra cui tastiera, mouse e schermo.

Tutti i trasferimenti di dati da CPU a dispositivi di I/O sono gestiti da appositi sistemi hardware o software specializzati nel controllo di quel specifico dispositivo o di quella specifica categoria di dispositivi.

Device Controller (Hardware) Consiste in un effettivo microchip integrato sulla scheda madre o alloggiato su una scheda di espansione che controlla direttamente il dispositivo fisico.

Device Driver (Software) Si tratta di un modulo del sistema operativo che conosce la sintassi e la logica per comunicare correttamente con il dispositivo di I/O.

Una questione fondamentale nella gestione delle periferiche riguarda il meccanismo con cui il processore interagisce con l'hardware: *come legge e scrive la CPU nei registri del device controller?*

Per rispondere a questa domanda si distinguono principalmente due approcci architetturali:

1. **Port-Mapped I/O (PMIO)**: Prevede l'utilizzo di **istruzioni CPU dedicate** (come ad esempio le istruzioni `IN` e `OUT` nelle architetture x86). Questo approccio nasce dal fatto che, a livello di *control bus*, le consuete operazioni di `LOAD` e `STORE` possono essere utilizzate solo se l'indirizzo di destinazione fa riferimento a una locazione di memoria principale. Si rende quindi necessaria la definizione di uno spazio di indirizzamento separato e di istruzioni specifiche riservate alla comunicazione con i dispositivi di I/O.

2. **Memory-Mapped I/O (MMIO)**: Rappresenta la soluzione standard e moderna, che consiste nella predisposizione di uno spazio di memoria e dei registri interamente dedicati a questi dispositivi.
   Un controller espone tipicamente tre classi fondamentali di registri:

   - **Status register**: registro di sola lettura, riporta lo stato corrente del dispositivo (BUSY, READY, ...).
   - **Command register**: sola scrittura, indica al dispositivo *cosa* fare.
   - **Data register**: lettura e scrittura, contiene i dati inviati/ricevuti al/dal dispositivo.

1.7.1 Protocolli di trasferimento I/O

Una delle problematiche fondamentali dell'architettura di un elaboratore riguarda la modalità con cui la CPU coordina e gestisce il trasferimento dei dati da e verso un dispositivo di Input/Output. Per rispondere a questa esigenza, si adottano principalmente tre protocolli:

1. **Programmed I/O (Polling)**: la CPU controlla ciclicamente lo stato del dispositivo per verificare se è pronto a trasferire i dati. Tale approccio può avere senso ed essere efficiente quando si devono trasferire pochissimi dati, in quanto evita l'overhead legato alla gestione delle interruzioni.

2. **Interrupt-Driven I/O**: come già accennato durante l'analisi del *control bus*, in questo schema è il dispositivo a notificare alla CPU l'avvenuto completamento di un'operazione tramite un segnale di interruzione. Risulta una modalità nettamente meno onerosa rispetto al polling quando si devono trasmettere quantitativi di dati più consistenti. In particolare, appena la CPU riceve questo segnale di interruzione, effettua una "fotografia" dello stato corrente dei registri e passa all'esecuzione di una particolare *routine* (**Interrupt Service Routine (ISR)**). Nello specifico, tutte le routine sono memorizzate all'interno di una struttura dati detta **Interrupt Vector Table** a cui la CPU ha accesso e che contiene i puntatori ad ogni specifica ISR.

3. **Direct Memory Access (DMA)**: un modulo hardware dedicato si occupa di trasferire i dati direttamente tra la memoria principale e il dispositivo di I/O, senza che la CPU debba intervenire per ogni singolo byte, riducendo drasticamente il carico di lavoro del processore.

2

Introduzione ai Sistemi Operativi

2.1 Cos'è un sistema operativo?

Definizione 2.1.1: Sistema Operativo (OS)

Un **sistema operativo** (Operating System, OS) è una collezione di software di sistema che gestisce le risorse hardware e fornisce servizi comuni per i programmi applicativi.

Il nucleo del sistema operativo è il primo software che si avvia all'avvio del computer.

2.2 I livelli di astrazione forniti dal SO

Il sistema operativo abbiamo capito essere come una sorta di "autorità" che gestisce tutte le risorse hardware e software per permettere all'utente di utilizzare la macchina nel modo più semplice e intuitivo possibile, avendo più o meno controllo sulle risorse (a seconda del sistema operativo che vogliamo implementare).

In generale, un buon sistema operativo deve:

Fornire l'astrazione adeguata , ovvero semplificare l'esecuzione di alcune operazioni, come ad esempio l'accesso al filesystem mediante una struttura organizzata in file e cartelle. Chiaramente questa struttura è puramente logica: se non venisse implementato questo tipo di astrazione, saremmo costretti ad accedere ai nostri file indicando il disco rigido e il settore in cui questo è contenuto.

Gestire in modo efficiente le risorse , decidendo quante e quali risorse allocare per ciascun processo in esecuzione alla CPU, e impedire che un processo prevalga sugli altri (*starvation*).

Controllare l'esecuzione delle applicazioni , impedendo ad esempio che un processo acceda (involontariamente) allo spazio di memoria riservato ad un altro processo, oppure impedendo che un programma generante un errore mandi in crash l'intero sistema.

Gestire l'esecuzione concorrente di più processi , suddividendo ogni processo in *thread* differenti (letteralmente dei sotto-programmi in esecuzione) e dando l'illusione che esistano contemporaneamente più CPU differenti che eseguono ognuna un processo (la magia del **timesharing**, di cui si parlerà più avanti).

2.3 Processi e Thread

Nella scorsa sezione abbiamo spesso citato i processi, senza mai darne una specifica definizione. La lasciamo di seguito.

Definizione 2.3.1: Processo

Un **processo** è un'**istanza di un programma in esecuzione**. Quando noi scriviamo un codice python, stiamo scrivendo un *programma*, qualcosa che può essere eseguito (ma possiamo pensare analogamente con tutti i software che installiamo da internet).
Una volta che manderemo in esecuzione tale codice o tale programma, per la CPU quello diventerà un processo.

Sempre come anticipato prima, diamo anche una definizione di Thread.

Definizione 2.3.2: Thread

Un **thread** è la più piccola unità di esecuzione indirizzabile dal sistema operativo, e consiste sostanzialmente in un *flusso di esecuzione* di un singolo processo. Detto con parole più semplici, possiamo suddividere un singolo processo in più thread, ognuno che si occupa di eseguire una parte di quel processo.

Da questa definizione si evincono due importanti caratteristiche dei thread:

- Possono essere creati **più thread diversi** relativi allo stesso processo.
- Più thread dello stesso processo condividono lo **stesso spazio di memorie per codice e variabili**.
- Dato che ogni thread esegue una *porzione diversa* dello stesso processo, ovviamente non possono essere condivisi registri come il Program Counter e i registri di calcolo.

2.4 La virtualizzazione della memoria

Fisicamente, la memoria centrale RAM è una griglia di celle in cui, per ogni cella, possiamo memorizzare un dato di una certa lunghezza (tipicamente equivalente alla *lunghezza di word* dell'architettura). Nell'esecuzione dei programmi, ogni programma può memorizzare le proprie variabili in memoria, tuttavia il SO si occupa di **virtualizzare** tale processo, assegnando ad ogni processo un indirizzo di memoria "finto", che verrà poi mappato ad un indirizzo reale in RAM. Ad esempio:

- L'indirizzo viruale `0x1000` nel Processo A viene mappato all'indirizzo fisico `0x7fff000`.
- Lo stesso indirizzo logico `0x1000` nel Processo B può essere invece tranquillamente mappato all'indirizzo fisico `0x8aaa000`.

A garantire che due indirizzi virtuali uguali non mappino allo stesso indirizzo fisico in memoria ci pensano il SO e la **Memory Management Unit (MMU)**.

2.5 Perché il Sistema Operativo necessita di supporto hardware

Il Sistema Operativo è, in fin dei conti, semplicemente un software. Se venisse eseguito sulla CPU esattamente allo stesso modo di un'applicazione utente, nulla impedirebbe a un programma utente di:

- Scrivere codice che disabiliti la protezione della memoria del Sistema Operativo;
- Leggere file personali appartenenti a un altro utente;
- Disabilitare gli interrupt e monopolizzare la CPU a tempo indeterminato.

Risposta? Nulla. Di fatto, se un sistema operativo non sfruttasse a pieno le risorse hardware, un utente potrebbe tranquillamente fare tutte queste cose.

2.5.1 Esecuzione in due modalità

Da questo problema nasce dunque la necessità di creare una barriera logica su cosa può e cosa non può fare un programma utente. Nasce così la **Dual-Mode Execution**, che definisce una gamma di istruzioni riservate all'esecuzione solo da parte del Kernel del sistema operativo.

Come viene implementata questa problematica? Definendo uno speciale bit, detto **Mode Bit**, memorizzate all'interno di uno speciale registro della CPU (tipicamente il PSR, Program Status Register).

- Se tale bit assume valore 0, la CPU eseguirà l'istruzione in **Kernel Mode** (tutti i privilegi).
- Altrimenti, con bit valore 1, l'istruzione verrà eseguita in **User Mode**, dunque con privilegi limitati.

Osservazione 2.5.1: E' lo stesso dell'"Esegui come amministratore"?

Anche se si potrebbe pensare ad un'analogia con quando ci viene chiesta l'autorizzazione dell'amministratore del nostro computer per eseguire un programma, per tali situazioni ci troviamo già ad un livello di astrazione più alto, dove ci serve la conferma di **un altro utente** e **non** del *sistema operativo*.

In generale, la transizione Kernel Mode -> User Mode si articola secondo i seguenti passaggi:

1. Viene memorizzato lo stato attuale dei processi e dei loro registri.
2. Viene effettuata la transizione a Kernel Mode.
3. Viene eseguita l'istruzione di sistema apposita.
4. All'interno della codifica di ogni istruzione di sistema è presente anche un blocco di istruzioni che si occupa di **ripristinare lo stato precedente alla kernel mode**, **mettere ad 1 il Mode Bit** e **ritornare alla normale esecuzione del processo**.

Alcuni esempi di istruzioni riservate alla Kernel Mode sono ad esempio `HLT`, che "spegne" la CPU, oppure `CLI/STI` che disabilita l'invio dei segnali di interrupt da parte di dispositivi hardware esterni. Ma anche banalmente l'accesso ai file e alle directories è un'operazione da Kernel Mode. Ci chiediamo dunque: come è possibile che noi riusciamo a fare queste operazioni in modo così semplice, nonostante siano formalmente riservate al sistema?

Ciò avviene principalmente secondo tre modalità differenti.

2.5.1.1 Chiamate di sistema

Il sistema operativo offre una lista di funzionalità Kernel Mode a cui l'utente può fare richiesta per eseguire una determinata operazione. Prendendo il precedente esempio dell'accesso al filesystem, quando clicchiamo sull'icon dell'`Esplora file`, implicitamente stiamo effettuando una chiamata al sistema operativo per permetterci di accedere ai file sul disco (quelli a cui l'utente è autorizzato, chiaramente). Come già detto, sta al sistema operativo fornire queste funzionalità, e dunque ci potrebbero essere sistemi operativi più o meno permissivi in tal senso.

Nell'ottica delle chiamate di sistema, ogni sistema operativo definisce una array detto **System Call Translation Table**, dove ogni elemento consiste in un puntatore ad una specifica funzionalità di sistema.

Gli argomenti di una syscall (pensiamo all'esempio di prima dell'accesso ad un file mediante una funzione `open`: dobbiamo chiaramente passare il percorso del file come argomento) possono essere memorizzati mediante:

- Registri, in caso di argomenti molto piccoli.
- Un stack di memoria esclusivamente adibito a questa funzionalità.
- Lo stack dell'utente.

Per quanto utili e potenti, le chiamate di sistema sono ouno strumento estremamente oneroso da implementare, in quanto richiedono un salvataggio e ripristino dello stato dei registri per ogni chiamata. Ecco perché molti linguaggi di programmazione ottimizzano multiple chiamate alla stessa funzione accorpandole, quando possibile, in una sola.

Osservazione 2.5.2: Esempi di syscalls

Funzioni come il `printf` nel linguaggio C, oppure il `System.out.println` in Java di per sé non sono delle vere e proprie syscalls. Tuttavia, se andassimo a leggere il codice mediante le quali sono state implementate queste funzionalità, ci accorgeremo di come esse facciano a capo ad una opportuna funzionalità di sistema che permette di mandare a schermo una serie di caratteri.

2.5.1.2 Eccezioni

Sono gli errori che ci vengono lanciati da un programma e che ne causano il crash. Anziché mandare in crash tutta l'applicazione, il sistema operativo deve gestire tali anormalità per impedire l'interruzione del lavoro di tutta la macchina. Sono comportamenti prevedibili e dunque **sincroni**, in quanto provengono da determinate istruzioni di un programma. Per fare un esempio banale, se il compilatore vede che è presente un'operazione di casting di tipo, è molto probabile che se il tipo passato non sia conforme quest'operazione lanci un'errore. Dunque, per quanto spesso siano imprevedibili, il sistema operativo e più in particolare i compilatori dei linguaggi possono attuare delle strategie per "prevedere" tali comportamenti. Tuttavia non tutti i tipi di eccezioni sono correttibili: alcune, dette **aborts**, causano errori hardware o di sistema irrecuperabili.

2.5.1.3 Interruzioni hardware

Come visto precedentemente, si verificano quando un dispositivo esterno di I/O interrompe il processo che stava eseguendo, e dunque invia un segnale di `INTERRUPT` al sistema operativo. Questo tipo di modalità è **asincrona**, in quanto non si potrà prevedere in alcun modo quando un dispositivo esterno abbia completato la propria operazione (es: quando il pacchetto è arrivato a destinazione).

La gestione da parte della CPU degli eventi asincroni legati ad interruzioni ed eccezioni avviene mediante un'array chiamato **Interrupt Descriptor Table (IDT)**. Ogni evento asincrono viene è marcato dal sistema operativo con un numero da 0 a 255. Questa marcatura stabilisce una corrispondenza biunivoca tra l'evento in sé e l'indice della IDT in cui si trovano le istruzioni necessarie a processare quell'evento.

A volte potrebbe accadere che il Kernel abbia bisogno di eseguire codice senza venire interrotto. Ecco perché la CPU mette a disposizione istruzioni come `CLI` per ignorare i flag di interrupt e `STI` per riabilitarne la lettura. Tuttavia non tutti gli interrupt possono essere ignorati: da qui la divisione tra **Maskable Interrupts** (ignorabili) e **Non-Maskable Interrupts** (critici, non ignorabili).

2.5.1.4 Timer di interruzione

Un esempio classico di interruzione che non può in alcun modo essere ignorata riguarda il **Timer Interrupt**.
Come suggerisce il nome, si tratta di un effettivo dispositivo hardware che interrompe periodicamente la CPU in modo da impedire l'esecuzione infinita o prioritaria di alcuni processi. Senza tale dispositivo, un programma contenente ad esempio `while (true) ...` verrebbe eseguito all'infinito. Per fortuna non è così, e grazie a questo timer di interruzione alcuni programmi esauriscono il proprio "tempo a disposizione" e vengono definitivamente spenti.

2.6 Protezione degli accessi in memoria

Il sistema operativo ha anche il compito di garantire che ogni processo esegua le proprie istruzioni e memorizzi le proprie variabili all'interno di uno spazio di memoria ben definito. Oltre a garantire che non si vadano ad utilizzare aree di memoria riservate ad altri processi, è fondamentale impedire che un processo possa accedere ad aree di memoria riservate al Kernel.

Questa richiesta viene semplicemente risolta impostando degli effettivi limiti per gli indirizzi di memoria accessibili da un singolo processo.
Ad esempio, supponiamo di avere tre attori in gioco:

- Spazio di memoria kernel, che va da `0000` a `0999`
- Spazio di memoria Processo A, che va da `1000` a `1499`.
- Spazio di memoria processo B, che va da `1500` a `1999`.

Nel momento in cui il Processo A genererà un indirizzo in cui memorizzare una variabile o in cui accedere ad una variabile, la CPU dovrà fare un semplice controllo: se tale indirizzo non risiede nel limite di indirizzi riservati al Processo A, quell'operazione va bloccata.

2.6.1 Memory Management Unit (MMU)

La Memory Management Unit (MMU) è quella **componente hardware che si occupa di tradurre gli indirizzi virtuali assegnati ai processi in indirizzi fisici nella memoria centrale**. Sostanzialmente, è tutto grazie alla MMU se i nostri processi riescono a coesistere senza intaccare l'uno nell'altro. La MMU è il componente hardware, che però si occupa di leggere le varie corrispondenze contenute all'interno di una particolare struttura chiamata **Page Table**. Ogni valore della page table (**Page Table Entry (PTE)**) contiene vari campi, tra cui l'indirizzo di memoria della singola entrata, i permessi di accesso (lettura/scrittura), il livello di autorizzazione (kernel mode/user mode).

Osservazione 2.6.1: Come funziona con il DMA?

Precedentemente avevamo introdotto il DMA, ovvero quel dispositivo hardware che si occupa di gestire tutti i segnali di interruzione relativi ai dispositivi di I/O. La domanda sorge spontanea: dove sono memorizzati i registri di questo controller? L'utente avrebbe modo di accedervi?

I registri vengono mappati ad opportune porte di I/O, e il mapping di queste porte avviene sempre grazie alla MMU che imposta tali aree di memoria come "Kernel Mode", dunque inaccessibili all'utente.

3

Il linguaggio di programmazione C

Perché studiare il linguaggio di programmazione C in un corso di sistemi operativi? Oppure, più in generale, perché studiarlo proprio? La risposta è semplice: C è il linguaggio più utilizzato al mondo, quello con cui sono stati scritti Windows, Linux, e quasi tutti i sistemi operativi. Si tratta di un linguaggio strettamente a basso livello (non quanto l'assembly chiaramente), che permette al programmatore di gestire la memoria del suo programma, tramite allocazioni e deallocazioni manuali. Nelle seguenti sezioni, partiremo davvero dalle basi del linguaggio. Questo capitolo non vuole essere sostituitivo ad un eventuale libro dedicato, ma fornire solo le conoscenze di base per l'uso del linguaggio e di eventuali esempi futuri.

3.1 Le variabili in C

Come ormai dovrebbe essere familiare a molti, una **variabile** è una locazione di memoria a cui abbiamo attribuito un nome, e in cui possiamo memorizzare dei dati. Un esempio di dichiarazione di variabile nel linguaggio C è il seguente `int numero = 42;`.

Come vediamo, la sintassi è la stessa che in altri linguaggi: tipo della variabile, nome, operatore di assegnazione, valore effettivo (se vogliamo attribuirlo da subito).
Infatti, il processo di dichiarazione di una variabile può essere suddiviso in tre momenti:

- **Dichiarazione**: quando esplicitiamo semplicemente il tipo e il nome della variabile, senza darle alcun valore. Esempio: `int numero;`
- **Inizializzazione**: può avvenire solo dopo la dichiarazione, e consiste nell'attribuire un valore ad una variabile già esistente. Esempio: `numero = 4;`
- **Definizione**: combina i due passi precedenti, dichiarando una variabile e inizializzandola ad un certo valore. Esempio: `int numero = 0;`

3.1.1 Visibilità delle variabili

Nel linguaggio C, le variabili hanno due livelli di visibilità:

- **Visibilità locale**: quando una variabile viene definita all'interno di una funzione, può essere vista e utilizzata **solo** da comandi presenti all'interno di quella funzione.

```
1       void esempio() {
2         int x = 3;
3         printf("%d", x)
4       }
5
6       printf("%d", x) // ERRORE!
```

- **Visibilità globale**: si tratta di una pratica sconsigliata, in genere, ma consiste nel memorizzare le variabili fuori da ogni funzione in modo che queste siano accessibili da qualunque pezzo di codice all'interno del file.

```
1       int x = 0;
2
3       void esempio() {
4         printf("%d", x) // CORRETTO, definita globalmente.
5       }
```

I livelli di visibilità sono qualcosa di automatico, ma i programmatori possono decidere di consigliare al compilatore dove memorizzare una data variabile:

- `register`: consiglia al compilatore di memorizzare la variabile all'interno di un registro, in quanto (probabilmente) ne sarà fatto frequente uso.
- `volatile`: indica che il valore di questa variabile potrebbe cambiare per nature indipendenti al programma in cui è stata definita.

3.1.2 I tipi di variabili

Molto velocemente, di seguito si indicano i tipi di variabili fondamentali del linguaggio, cosa permettono di memorizzare e quanto spazio in memoria occupano.

Tipo	Byte occupati	Cosa memorizza
char	1	Caratteri codificati in ASCII
short	2	Numeri interi con segno nel range [-32.768, 32767]
int	4	Numeri interi con segno
unsigned int	4	Numeri interi senza segno
float	4	Numeri con la virgola con una precisione "ridotta"
double	8	Numeri con la virgola e ampia precisione
bool	1	Valori booleani che memorizzano valgono 0 o 1

Inoltre in C, come in altri linguaggi, è presente il **casting di tipo**, ovvero la conversione tra tipi diversi.

- **Casting implicito**, dove una variabile di un tipo viene convertita da sola in un altro. Ad esempio `double d = 30.4; int i = d;`
  In questo caso una variabile di tipo `double` viene messa come valore ad una variabile dichiarata come `int`, il che vuol dire che viene persa nella rappresentazione intera.
- **Casting esplicito**, dove diciamo esplicitamente al compilatore cosa convertire in cosa. Esempio: `double pi = 3.14; int ipi = (int) pi;`

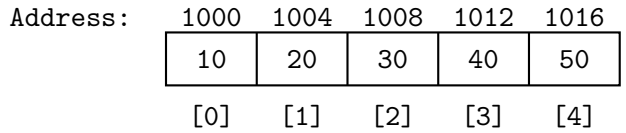
3.2 Gli Array

Un Array è una **sequenza di dati tutti dello stesso tipo** (omogenei tra loro). L'accesso agli elementi viene effettuato mediante `indici`: ogni indice punta ad un determinato elemento. Supponendo di avere un array di `n` elementi, gli indici vanno da `0` (primo elemento) a `n-1` (ultimo elemento). Un Array si può dichiarare come segue:

- `int array[5]`: definizione dell'array, bisogna obbligatoriamente indicare il **tipo di ogni elemento** e il **numero totale di elementi**.

- `int array[] = {1,2,3}`: dichiarazione dell'array con inizializzazione. Quando inizializziamo un array possiamo anche omettere la dimensione, in quanto il compilatore li va a contare.

In memoria, un Array è rappresentato come sequenza di locazioni di memoria, ognuna occupante un numero di byte corrispondente al tipo dell'array.



`sizeof(arr)` = 5 × 4 = 20 bytes

Giusto per accennarlo, si possono definire anche array bidimensionali, dove ogni elemento di un array bidimensionale è a sua volta un array. Un array bidimensionale viene spesso visto come una matrice di `n` righe ed `m` colonne

```
int matrice[2][3] = {{1,2,3}, {4,5,6}}
```

E come si può immaginare, per accedere ad una cella della matrice, si deve indicizzare sia per riga che per colonna. Ad esempio per ottenere l'elemento `4` devo scrivere `matrice[1][0]`.

3.3 Le stringhe

Al contrario di altri linguaggi, come ad esempio il Python o Java, le stringhe non sono un tipo nativo in C. Queste sono definite come un array di caratteri, che ha come ultimo carattere il carattere di escape `'\0'`, che indica il *fine parola*. La definizione di una stringa avviene dunque come segue:

```
char saluto[] = "Ciao";
```

E tale stringa sarà rappresentata in memoria esattamente come abbiamo visto rappresentare un'Array, con l'aggiunta del carattere '\0' citato prima. In questo caso avremo dunque

```
['C', 'i', 'a','o','\0']
```

3.3.1 Funzioni per le stringhe

Il linguaggio C mette a disposizione una libreria `string.h` che dispone anche di alcune funzioni utili nel lavorare con le stringhe.

- **Controllare la lunghezza di una stringa**: si utilizza `strlen` al contrario del classico metodo `sizeof` utilizzato per tutti gli altri tipi di variabile. Questo in quanto `sizeof` conta anche il carattere di fine riga.
- Per **copiare una stringa** all'interno di un'altra possiamo usare la funzione `strcpy`, che però non effettua alcun controllo sulla lunghezza della stringa di destinazione e su eventuali overflow. Il metodo consigliato è dunque `strncpy`, considerato più sicuro in quanto implementa tali controlli.
- Per **confrontare due stringhe** si utilizza il metodo `strcmp`, che prende come parametri due stringhe e restituisce `0 se le stringhe sono uguali`, `1 altrimenti`.

3.4 I puntatori

Uno dei punti di forza, se non *il* punto di forza del linguaggio C è proprio la flessibilità riguardo al controllo sulla memoria. Gran parte di questa gestione è garantita anche grazie ai **puntatori**, ovvero delle **variabili il cui valore è un indirizzo di memoria**. La dicharazione è molto semplice: se abbiamo una variabile di tipo `int` e vogliamo dichiarare un puntatore a quella variabile intera, lo faremo semplicemente aggiungendo un asterisco accanto al nome del tipo, dunque `int*`. Vale ovviamente per ogni tipo di variabile. Chiaramente bisogna mantenere **coerenza** tra il tipo della variabile che vogliamo puntare e il tipo del puntatore.

Strettamente legato al concetto di puntatore è il concetto di indirizzo. Per prendere l'indirizzo appartenente ad una variabile utilizziamo l'operatore `&`.

```
int x = 42;
int* p = $x
```

Inoltre, se utilizziamo di nuovo l'operatore `*` davanti ad un puntatore, faremo riferimento al **valore contenuto nell'indirizzo di memoria puntato da quel puntatore.**

```
int x = 42;
int* p = $x
printf("%d", *p) // Out: 42
```

Volendo, possiamo inizializzare un puntatore al valore speciale `NULL`, per indicare che non punta a nulla.

Osservazione 3.4.1: Puntatori `void*`

In alternativa a quanto visto prima, se vogliamo definire un puntatore che nel corso del programma può fare riferimento a tipi di variabili differenti (non è una good practice, però), potremmo definire un puntatore come `void*`. In questo modo il nostro puntatore potrà puntare all'indirizzo di memoria di qualsiasi variabile, tuttavia **non** potremmo usare l'operatore `*` per accedere al valore dell'indirizzo puntato da quel puntatore. Dovremo fare un casting esplicito al tipo della variabile puntata.

```
int x = 42;
void* p = &x
printf("%d", (int*)*p)
```

Possiam anche definire dei puntatori **costanti**, ovverosia che possono cambiare il valore (cioè l'indirizzo) memorizzato al proprio interno, ma **non può cambiare il valore** associato a quell'indirizzo.

3.4.1 Aritmetica dei puntatori

Di seguito alcune nozioni importanti su come funzionano i puntatori con gli Array e il tipo di operazioni aritmetiche che possonoe essere svolte sui puntatori.

- Un puntatore p ad un Array A punterà **sempre** al primo elemento di quell'array (dunque `int* p = arr`) è come dire `int* p = &arr[0]`.
- Quando incrementiamo un puntatore di un'unità, il valore dell'indirizzo aumenta della `sizeof` del tipo della variabile puntata. Ad esempio, se abbiamo un puntatore `double* p = &x` e facciamo `p++`, il valore dell'indirizzo di memoria andrà avanti della `sizeof(double)`, ovvero il numero di byte richiesti in memoria per memorizzare un double.
  Questo può essere un modo alternativo per iterare lungo gli elementi di un Array.
- Sottraendo il valore di due puntatori `p1, p2` (gli indirizzi di memoria, dunque), otteniamo come risultato **il numero di elementi compresi tra p1 e p2**.
  Attenzione! Non la dimensione complessiva di quegli elementi, ma proprio il numero di elementi in mezzo! Ad esempio:

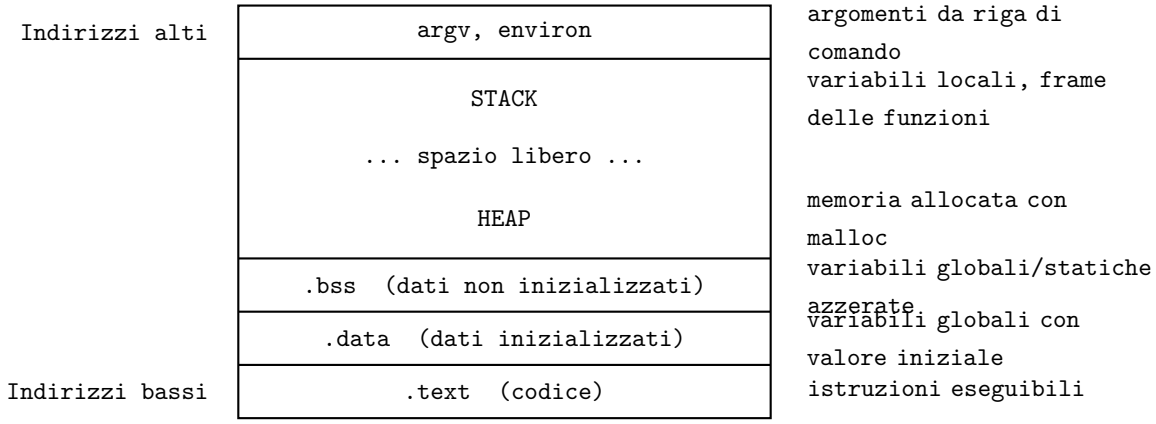
```
int arr[5] = 1,2,3,4,5
```

```
int* secondo = &arr[1]
 int* ultimo = &arr[4]
\\ ultimo - secondo = 2
```

- Possono essere effettuate operazioni di comparazioni tra puntatori, e funzionano come possiamo aspettarci.

3.5 La gestione della memoria

Ogni programma scritto in C organizza la propria memoria nel modo descritto dal diagramma sottostante.



In questa sezione andremo a discutere, più o meno nel dettaglio, il funzionamento di queste componenti.

3.5.1 Lo Stack

Lo **Stack** è una regione di memoria utilizzato principalmente per la memorizzazione delle funzioni e delle variabili a loro associate (che siano essi parametri passati alle funzioni o variabili locali dichiarate al loro interno). Come suggerisce il nome, questa memoria opera in modalità **LIFO**, ovvero l'ultima funzione ad essere inserita sulla pila è anche la prima ad uscire (intuitivamente, se abbiamo una funzione A che chiama al suo interno una funzione B, la funzione B verrà terminata prima della sua chiamante, e dunque rimossa prima dalla pila).

Ogni chiamata di funzione crea un cosiddetto **stack frame**, ovvero un blocco di celle di memoria in cui vengono memorizzati dei dati riguardanti la funzione:

- **Gli argomenti passati alla funzione** che, se sono pochi, vengono memorizzati all'interno di speciali registri, altrimenti vengono inseriti nello stack frame.

- L'indirizzo della prossima istruzione da eseguire **dopo** la fine dell'esecuzione della funzione. Questo valore viene memorizzato in quanto anche la funzione chiamata dovrà seguire un suo valore di Program Counter.
- Un puntatore, detto **frame pointer** che punta all'inizio dello **stack frame della funzione chiamante**.
- **Variabili locali** alla funzione.

Inoltre, nella gestione della memoria Stack, vengono utilizzati due registri:

- **Stack pointer**, che punta alla cima dello stack.
- **Frame pointer** che, come già detto, punta ad uno specifico stack frame.

Come si potrebbe intuire, un eccessivo numero di chiamate di funzione può portare al **riempimento eccessivo dello stack**. In quel caso si parla di **Stack Overflow**, e può accadere ad esempio quando si hanno troppe chiamate ricorsive ad una funzione.

Osservazione 3.5.1: Il compilatore e lo Stack

Quando un programma viene compilato, il compilatore analizza ogni singola funzione, con i suoi parametri, variabili locali e tipi di ritorno e conosce già la dimensione dello stack frame associato a quella funzione.

3.5.2 Lo Heap

Lo **Heap** è una regione di memoria dedicata all'allocazione dinamica e **manuale** delle variabili da parte dell'utente. A differenza dello Stack, la dimensione dello Heap non è conosciuta a compilazione, ma rappresenta comunque un'area di memoria molto vasta.
Le variabili in C possono essere manualmente allocate tramite appositi comandi, che permetteranno alle variabili di essere memorizzate nell'Heap e non in altre aree di memoria.
I comandi predisposti sono i seguenti:

`malloc(n):` serve ad allocare manualmente un blocco di `n` bytes in memoria. Ad esempio, se volessimo allocare dinamicamente un array di 5 elementi potremmo scrivere `int* arr = malloc(5 * sizeof(int))`.

`calloc(n, size):` serve ad allocare `n` byte, ognuno di dimensione `size` e di inizializzare tutti quei byte al valore di default per quel tipo di dato. Tenendoci sull'esempio dell'Array di prima, se facessimo `int* arr = calloc(5, sizeof(int))` e provassimo a mandare a schermo l'Array otterremmo `[0, 0, 0, 0, 0]`.

`realloc(ptr, new_size)`: serve per riallocare un blocco di memoria precedentemente allocato. Supponendo di aver definito con `malloc` prima un array `arr` di 5 elementi interi, ma ci siamo resi conto che ci serve spazio per almeno 8 elementi, possiamo semplicemente fare `int* tmp = realloc(arr, 8 * sizeof(int))`.

`free(ptr)`: fondamentale, serve a liberare un puntatore di memoria precedentemente allocato manualmente. Importantissimo è anche impostare il puntatore a `NULL` dopo averlo liberato.

Allocando e deallocando frequentemente nella memoria Heap si potrebbe verificare la cosiddetta **frammentazione della memoria**, ossia avere complessivamente molta memoria libera, ma a blocchi separati da memoria utilizzata. La seguente figura renderà bene il concetto:

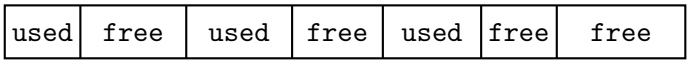


Figura 3.1: Frammentazione della memoria nell'Heap.

3.5.3 Errori di memoria

Spesso è proprio la libertà con la gestione della memoria conferitaci dal linguaggio ad essere causa della maggior parte degli errori che si incontrano nei programmi. Esempi degli errori più classici possono essere:

Perdita di memoria: si verifica quando ci dimentichiamo di liberare una locazione di memoria ormai inutilizzata.

Doppia chiamata di `free()`: si verifica quando chiamiamo `free()` due volte consecutive sullo stesso puntatore, senza averlo prima impostato a `NULL`.

Usare un puntatore dopo averlo liberato

Puntatori fantasma: si verifica quando ad un puntatore vengono restituiti valori di variabili locali a funzioni.

Puntatori non inizializzati: quando si dichiara un puntatore e non lo si inizializza a nessun valore, questo punta ad un indirizzo inutilizzato dal programma ma che potrebbe comunque servire in futuro. E' sempre buona pratica inizializzare i puntatori almeno a `NULL`.

Scrittura oltre i limiti di un Array: quando si prova ad accedere ad indici inesistenti di un Array (es: se `int arr[5]`, non esisterà l'elemento `arr[5]` in quanto il conteggio degli indici parte da 0).

Variabili inizializzate: una variabile non inizializzata contiene un qualsiasi valore presente precedentemente nello stack.

Esistono degli strumenti di compilazione come `valgrind` o `AddressSanitizer`, ma li citiamo solo per nozione e non per approfondirli.

3.6 La traduzione da sorgente ad eseguibile

Il processo di traduzione di un programma C nel suo eseguibile è un'operazione molto lunga e articolata in diverse fasi. In questa sezione vedremo quali sono queste sezioni e le discuteremo una per una. Di seguito un'immagine rappresentante la classica struttura di compilazione di un programma in C:

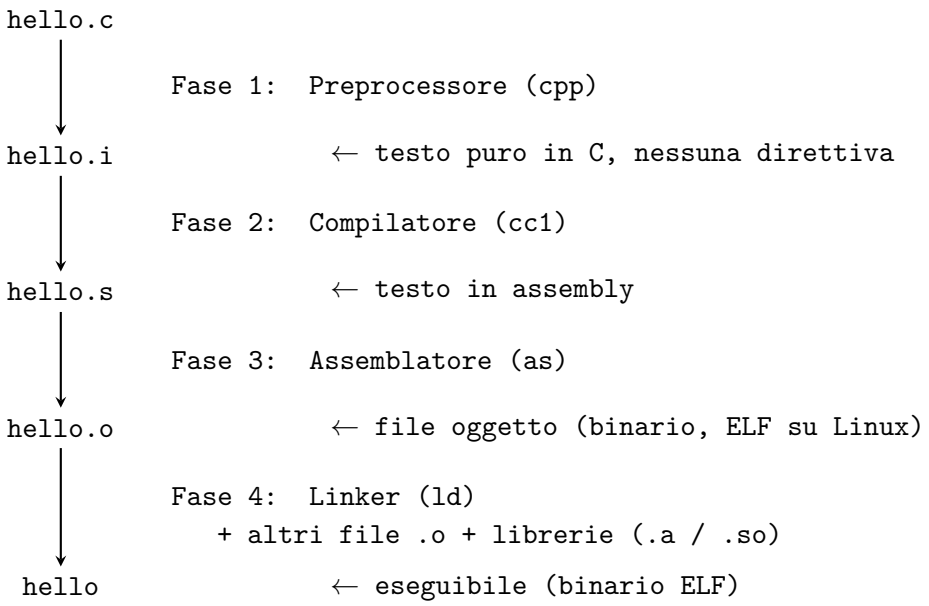


Figura 3.2: Pipeline di compilazione di un programma in C: dal sorgente (`.c`) all'eseguibile finale (`hello`) attraverso le quattro fasi fondamentali.

3.6.1 Il preprocessore

Si tratta del primissimo stadio dell'operazione di compilazione, e si occupa sostanzialmente di **manipolare il testo del programma** che abbiamo scritto. Parlando in modo più tecnico, si occupa di:

- Gestire tutti gli `#include`. Questo viene fatto **copiando e incollando** il codice del file incluso all'interno del nostro file.
- Sostituire tutti i `#define` con la corrispondente sostituzione scelta. Il comando `#define MAX 100` *alias* per una variabile all'interno del nostro programma. Nell'esempio fornito stiamo solo definendo un valore speciale `MAX` che vale 100, ma potremmo anche fare in modo che d'ora in poi nel nostro codice non dovremmo più dichiarare gli interi come `int` bensì come `intero` scrivendo semplicemente `#define intero int`.
- Rimozione dei commenti sostituendoli con un singolo spazio bianco.

Questa fase trasforma il file `hello.c -> hello.i`.

3.6.2 La compilazione

Si tratta della fase che si occupa di **tradurre il codice scritto in C nelle corrispettive direttive scritte in Assembly**. Anche se un progetto include più file, ogni file con estensione .c viene compilato separatamente. Questa fase trasforma il file `hello.i -> hello.s/asm`

3.6.3 L'assemblaggio

Fase in cui entra in gioco il cosiddetto **assembler**, che si occupa di **convertire il testo scritto in Assembly in istruzioni macchina**. Non entreremo nel dettaglio di quale corrispondenza viene scelta a livello di bit, ma ciò dipende dal tipo di architettura della macchina (CISC/RISC come visto nella sezione 1.4). Questa fase genera il file `hello.s/asm -> hello.o`.

3.6.4 Il linking

Si tratta della fase finale del processo di traduzione, e si occupa sostanzialmente di **unire tutti i file oggetto del progetto in un singolo file eseguibile**. In particolare, vengono fatte queste operazioni intermedie:

- **Risoluzione dei simboli:** se ad esempio nel file `main.c` chiamiamo una funzione `calcola(a, b)`, il linker si occuperà di cercare in tutti i file oggetto una funzione con quel nome. In caso non dovesse trovarla, solleverà un errore tipo `undefined reference to 'calcola'`.
- **Riallocazione:** unisce tutti i file di codice in una singola sezione `.text` e tutte le sezioni dati in una sezione `.data`.

Alla fine di questo processo di traduzione viene finalmente **generato il file eseguibile**.

3.6.5 Librerie statiche e dinamiche

Durante l'operazione di linking, possono essere generati due tipi diversi di librerie.

Libreria statica: si tratta di un file con estensione `.a` (che sta appunto per *archivio*) che contiene una collezione di file oggetto. Durante il processo di linking, *tutto* il codice di questa libreria viene copiato all'interno del programma principale.

Libreria dinamica: si tratta di un file con estensione `.so` pensato per essere condiviso tra più file diversi. Si tratta di un'ottima scelta quando non si vuole far crescere la dimensione dell'eseguibile. Il grande vantaggio sta nel fatto che la copia delle funzioni utili non avviene a compilazione, bensì quando una funzione viene chiamata (il linker si occupa solo di inserire dei riferimenti per ogni funzione e il file in cui questa è definita).

4

Processi e API di sistema

Abbiamo già definito il concetto di *processo* nei capitoli precedenti, ma in questo capitolo ne vedremo meglio la natura, come creare processi, come eseguirli, ecc...

4.1 L'astrazione dei processi

Un processo è un'istanza di un programma in esecuzione, che comprende i seguenti parametri:

- Il **codice** che deve essere eseguito.
- Il valore corrente del **Program Counter**.
- I contenuti dei **registri della CPU**.
- Uno **spazio di memoria virtuale** a cui accedere (ovverosia il suo stack, il suo heap, ...)
- Gli identificativi ai file su cui il processo deve lavorare.
- Un **identificativo univoco di processo**, chiamato **ProcessID (PID)**.

In base alla filosofia secondo cui viene progettato il sistema operativo, questo può decidere di procedere secondo due strategie per allocare il processo in RAM:

- **Caricamento immediato**: appena il processo entra in esecuzione, tutti i suoi dati vengono caricati in memoria RAM. Si tratta di una soluzione facile ma poco efficiente, in quanto potrebbero venir caricate in memoria anche porzioni di codice mai utilizzate.

- **Caricamento pigro**: una sezione del processo viene caricata solo quando deve essere effettivamente utilizzata. Entreremo più nel tecnico di questa modalità quando parleremo di paging e memoria.

4.2 Gli stati di un processo

Affinché lo scheduler del sistema operativo possa decidere quando mandare un processo in esecuzione, quando fermarlo e quando farne partire un altro, si attribuisce ad ogni processo uno **stato**.

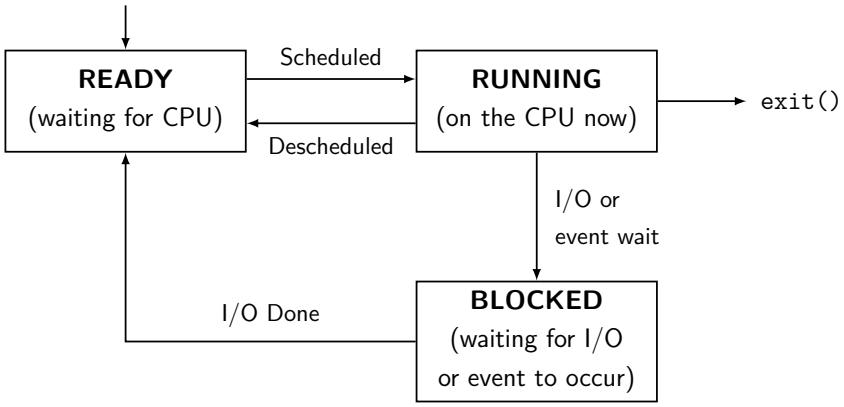


Figura 4.1: Diagramma a tre stati del ciclo di vita di un processo

Come si vede nella Figura 4.1 vi sono 3 possibili stati in cui un processo:

READY: i processi in questo stato sono pronti per essere eseguiti, e attendono solo che lo scheduler li mandi in esecuzione. Dire che un proceso è "pronto" per essere eseguito vuol dire che **possiede tutte le risorse necessarie per eseguire la prossima istruzione**.

RUNNING: è lo stato in cui si trova un **processo in esecuzione**. NOTA BENE: Ci può essere **solo un** processo in esecuzione alla volta su una singola CPU, in quanto è impossibile eseguire due processi simultaneamente. Se avessimo una CPU con n core, si potrebbero eseguire simultaneamente n processi, in quanto ogni core opererebbe come una CPU distinta con i suoi registri e i suoi stati.

BLOCKED: i processi in questo stato sono quelli che **attendono una risorsa da un dispositivo di I/O**, senza la quale non potrebbero continuare l'esecuzione del programma (es: un input da tastiera). Viene definita una categoria a parte per quando si richiedono risorse esterne in quanto abbiamo visto come l'interfacciamento con dispositivi esterni sia un'operazione molto lenta (soprattutto rispetto ai cicli di clock della CPU).

Osservazione 4.2.1: Transizione RUNNING → READY

Affinché un processo passi da READY a RUNNING è sufficiente che lo scheduler decida che "è arrivato il suo momento di eseguire".

Ma per il contrario? Se l'esecuzione di un processo fila liscia, senza neanche richiesta di interfacciarsi con dispositivi esterni, entra il gioco quello che nei capitoli precedenti avevamo chiamato *Timer Interrupt*, ovvero un timer di sistema che assegna ad ogni processo un certo intervallo di tempo per eseguire le sue istruzioni, dopodiché lo fa terminare per lasciare spazio ad altri processi.

4.3 Il Process Control Block (PCB)

Affinché il sistema operativo possa tenere traccia di tutti i dati relativi ad un processo, per ognuno di questi viene associata una particolare struttura dati chiamata **Process Control Block (PCB)**.
Questa struttura contiene informazioni fondamentali riguardo al processo, come ad esempio:

- Il suo **PID**.
- Il suo **stato corrente**, secondo la logica vista poco fa.
- Il **puntatore alla pagina di memoria riservata al process** (**pagetable**).
- Lo **stato e il contenuto dei registri della CPU** (**trapframe**).
  Siccome abbiamo una sola CPU che può eseguire un processo alla volta, quando un processo passa dalla fase di esecuzione ad una qualsiasi altra fase, dovrà memorizzare i valori dei registri nel momento in cui l'esecuzione è stata interrotta, in modo che quando venga ripresa questa non riparta dall'inizio ma dall'ultimo "checkpoint".

Tutti i PCB dei processi inizializzati vengono memorizzati all'interno di una struttura dati chiamata **Process Table** che **risiede in memoria kernel** (in quanto deve essere lo scheduler del sistema operativo a decidere chi eseguire). Lo scheduler andrà poi a scansionare questa tabella in ricerca di processi con stato READY da mandare in esecuzione.

4.4 La gerarchia dei processi

Nei sistemi UNIX, i processi formano un albero in cui la radice è il processo **systemd** con PID = 1. Questo processo non può essere killato neanche volendo, in quanto è strettamente legato al funzionamento del kernel e del sistema operativo.

La gerarchia dei processi è organizzata ad albero in quanto ogni processo può creare altri processi figli (con il comando `fork()`) e dunque ogni processo ha anche un padre (il cui valore si chiama **Parent ProcessID (PPID)**).

4.5 La funzione fork()

La funzione `fork()` è forse una delle funzioni più utili quando si va a lavorare con i processi, in quanto permette di **creare un nuovo processo**.
Più precisamente, ci permette di **duplicare il PCB del processo padre**, ovvero il PCB del processo chiamante. Quest'operazione ci permetterà di allocare un PCB (dunque senza bisogno di conoscere la natura della struttura dati, astrazione!) e poi di modificarlo quasi a piacimento.

Vediamo un esempio di utilizzo di `fork()` all'interno di un codice e commentiamolo:

```
1   int main() {
2     int pid = fork();
3
4     if (pid < 0) {
5       perror("fork failed");
6     } else if (pid == 0) {
7       printf("Processo figlio con PID: %d", getpid());
8     } else {
9       printf("Processo padre con PID: %d", getpid());
10    }
11  }
```

Analizziamo ogni singola istruzione di questo codice che, seppur breve, ci dovrebbe aiutare a capire il funzionamento di `fork()`.

1. **Chiamata alla funzione fork()**: come abbiamo già detto, avendo mandato il programma in esecuzione, quello sarà un processo, supponiamo con PID = 1234. Chiamando `fork()` all'interno di questo processo, stiamo dicendo al sistema di creare un nuovo processo (dunque un nuovo PCB) che sia sostanzialmente una **copia** del PCB del processo chiamante (che sarà il padre di questo nuovo processo).

2. **Cosa viene copiato nel PCB**: il PCB padre e PCB figlio avranno in comune l'area di memoria in cui è presente il codice binario da eseguire, ossia entrambi punteranno **allo stesso indirizzo di memoria**.
   Oltre a questo, non hanno altro in comune: avranno sì gli stessi valori delle variabili,

ma solo perché i valori dei registri saranno stati copiati e incollati anche nel PCB figlio. Il processo figlio avrà le proprie aree di memoria stack e heap, dunque se provassero a modificare la stessa variabile non riuscirebbero ad agire sullo stesso valore in quanto sarebbero logicamente indipendenti. E' molto importante sapere che al processo figlio, ovviamente, viene assegnato un nuovo PID. Supponiamo 1235 per il nostro esempio.

3. A questo punto, avviene una sezione critica: avendo copiato nel processo figlio lo stato di **tutti** i registri, sarà stato copiato anche il valore del Program Counter, e dunque processo padre e figlio riprenderanno **dallo stesso identico punto**.

4. Discutiamo ora il valore di ritorno della funzione `fork()`: prima di restituire un valore, viene eseguita la funzione e dunque creato il processo figlio. A questo punto, come abbiamo detto, i due processi condivideranno la stessa area di memoria per il codice ma non la stessa per le variabili. Qui il sistema fa una cosa interessante:

   - All'interno del processo figlio, il valore di ritorno della funzione sarà **categoricamente 0**. Si tratta di una convenzione stabilita per indicare che il processo corrente è figlio di uno già esistente.
   - All'interno del processo padre, la funzione `fork()` restituirà il PID del processo appena creato.

   Ma come è possibile che una singola funzione restituisca un valore diverso a due processi diversi?
   Questo avviene in quanto, dopo aver creato le aree di memoria dedicate al processo figlio, il valore di ritorno della funzione verrà messo per ogni processo all'interno di uno speciale registro della CPU. Avendo due registri diversi, uno per il padre e uno per il figlio, è possibile creare questa "voluta discrepanza" tra i valori di ritorno.

5. Una volta che la chiamata a `fork()` (supponiamo) è andata a buon fine, quindi, i possibili valori assumibili dalla nostra variabile intera sono **0 per il processo figlio** e $>$ **0 per il processo padre.**
   Questa è la ragione per cui inseriamo quei controlli all'interno del codice: **non avendo garanzia di chi partirà per primo con l'esecuzione**, e magari volendo separare tra le operazioni che può compiere il figlio e quelle del padre, l'unico modo è necessariamente controllare il valore restituito da `fork()`.

Come ci si può aspettare, la funzione di libreria C `fork()` nasconde sottobanco una chiamata di sistema alla funzione `sys_fork()`. Viene effettuata una chiamata di sistema in quanto abbiamo già detto che la Process Table in cui sono memorizzati i PCB dei vari

processi risiede in memoria kernel, e dunque è strettamente vietato far accedere quell'area di memoria da parte di un'istruzione utente.

Inoltre, come è stato già detto nella descrizione del flusso di esecuzione del processo `fork()`, è impossibile predirre chi, tra il padre o il figlio, verrà eseguito prima come processo. Tutto dipende dallo scheduler e dalla priorità attribuita ad ogni processo. E' dunque sconsigliato scrivere porzioni di codice *assumendo che un processo venga eseguito prima dell'altro.*

Un modo per forzare l'esecuzione del processo figlio prima del processo padre è quello di mettere "a dormire" il processo padre, usando la funzione di libreria `sleep()`.

```
int main() {
  int pid = fork();

  if (pid < 0) {
    perror("errore nella fork");
  } else if (pid == 0) {
    // ESEGUO OPERAZIONI FIGLIO
  } else  {
    sleep(1);
    // ESEGUO OPERAZIONI PADRE
  }
}
```

In questo modo stiamo forzatamente interrompendo per 1 secondo l'esecuzione del processo padre, tempo che per la CPU è enorme dato che esegue gli switch tra un processo e l'altro nell'ordine di nanosecondi. Questo modo è chiaramente molto grossolano, ed è una **bad practice** in quanto mette a dormire il padre anche se il processo figlio è stato eseguito per primo.

Prima di concludere il discorso sul comando `fork()`, vediamo come si comportano multiple chiamate della funzione all'interno di un programma.
Supponiamo di avere questo codice:

```
fork(); (1)
fork(); (2)
fork(); (3)
printf("Hello, World!");
```

Ragioniamo: ad ogni chiamata di fork, il numero di processi viene duplicato, in quanto un padre crea un processo figlio.
Alla chiamata (1), diventano 2 processi. Siccome l'esecuzione del codice per tutti i processi riparte dallo stesso punto grazie al valore del Program Counter, ciascuno dei due processi eseguirà anche la `fork()` del punto (2). Dato che avremo 2 processi ad eseguire una `fork()` e ognuno di questi genera 2 processi figli, alla fine di (2) avremo un totale di 4 processi. Come si può immaginare, ognuno dei 4 processi eseguirà un `fork()` duplicando il numero di processi, arrivando così ad 8 processi. Vediamo dunque come il numero di processi finali è dato da 2^n dove n è il numero di `fork()` eseguite.

Dunque se volessimo mandare in crash il sistema operativo, ora sapremmo come fare:

```
1  while (1) {
2    fork();
3  }
```

Listing 4.1: Codice per implementare la cosiddetta "fork bomb".

Questo ciclo continua a creare processi finché non viene raggiunto il limite di processi memorizzabili all'interno della Process Table, e dunque il sistema operativo va in crash.
Una possibile soluzione a questo problema sarebbe definire un limite massimo di processi generabili da un singolo utente.

4.5.1 La filosofia Copy-on-Write

In modo molto ingenuo, il comando `fork()` copierebbe l'intero spazio di memoria di un processo (che potenzialmente potrebbero essere anche diversi GB di memoria) in una nuova area riservata al processo figlio.
I sistemi operativi moderni evitano questa pratica (proprio per evitare sprechi di memoria) implementando la cosiddetta filosofia **Copy-on-Write**:

- Alla creazione del nuovo processo, l'area di memoria a lui attribuita non è una copia di quella del processo padre, bensì gli viene attribuito lo **stesso indirizzo di memoria** della memory page del padre. Possiamo quindi dire che fino ad un certo momento condivideranno letteralmente la stessa risorsa, dato che viene passato l'indirizzo di questa.

- Quando uno dei due processi prova a modificare la suddetta area condivisa, viene sollevata una **Page Fault** dalla Memory Management Unit. Solo a questo punto viene creata la copia in memoria della pagina, che includerà le modifiche apportare

dal processo che le ha richieste.
A questo punto avremmo dunque due pagine di memoria separate:

- La pagina di memoria "originale", alla quale punta il processo che **non** ha provato a modificarla.
- La pagina di memoria *copiata e modificata* dal processo che **ha** provato a modificarla.

4.6 La sincronizzazione dei processi con wait()

Come abbiamo detto e come si vuole ripetere, è impossibile determinare chi, tra il processo padre e quello figlio, riprenda l'esecuzione in seguito ad una `fork()`.
Supponendo che il processo figlio termini prima di quello del padre, la memoria dedicata al processo figlio **non verrà eliminata**, bensì verrà mantenuta fino alla fine del processo padre. Il processo figlio verrà poi messo in uno stato detto **ZOMBIE**.
Per fare in modo che questo problema non si presenti, il sistema operaivo mette a disposizione la funzione `wait()` che ci permette di fermare l'esecuzione del processo padre in attesa che i processi figli siano terminati.

Vediamo di seguito la firma del metodo `wait()`, commentiamola, e poi vediamo un esempio di utilizzo nel codice.

```
int wait(int *wstatus);
```

- La funzione restituisce un valore intero che, se l'esecuzione di `wait()` è andata a buon fine, rappresenta il PID del processo figlio che è terminato.
- Il parametro `int *wstatus` è un modo intelligente per far restituire alla funzione un altro valore.
  Siccome non siamo su Python e non possiamo restituire tuple di valori, il segreto sta nel passare il puntatore ad una variabile non inizializzata e poi ci penserà la funzione `wait()` al suo interno a modificarla, memorizzandoci lo stato con cui si è conclusa l'esecuzione del processo figlio.

Vediamo ora un esempio concreto in un pezzo di codice.

```
1  int main() {
2    int pid = fork();
3    if (pid == 0) {
4      printf("Sta eseguendo il figlio...");
```

```
5           exit(42); // Un generico codice di uscita
6         }
7
8         // Adesso siamo nel parent e dobbiamo attendere che
              finisca il child
9         int stato;
10        int finito = wait(&stato);
11
12        if (child_pid == -1) {
13          perror("wait");
14        } else {
15          printf("Arrivo qui solo dopo che ho finito.")
16        }
17      }
```

Come vediamo, il valore assunto dalla variabile `finito` rappresenta il valore del processo figlio terminato. Nell'ottica di voler far eseguire prima il figlio del padre, questo è l'approccio che si deve attuare.
Esiste anche la funzione `waitpid(int child_pid)` che consente al processo padre di attendere la terminazione dell'esecuzione **solo del processo figlio che ha PID uguale a quello passato come parametro della funzione**.

4.7 La funzione exec()

La funzione `exec()` è una procedura fondamentale in quanto ci permette di sostituire il codice eseguito da un processo con un altro file binario contenente altro codice.
Si tratta di una funzionalità molto utile, in quanto non risulta avere particolarmente senso far eseguire sia un processo che i suoi figli lo stesso file binario.

Osservazione 4.7.1: Successo di exec()

Se la funzione `exec()` viene portata a termine con successo, il processo figlio non tornerà mai ad eseguire il processo padre in quanto avrà cambiato totalmente file binario.

Una variante di `exec()` è la funzione `execvp(const char* file, char* const argv[])`, che prende come input il file binario di cui eseguire il codice ed eventuali argomenti che servono a tale programma per completare la sua esecuzione.
Prendiamo ad esempio il seguente codice:

```
    char* args[] = {"ls", "-l" "-a", "\tmp", NULL};
    execvp("ls", args);
```

Listing 4.2: Il comando ls serve a listare il contenuto della directory "\tmp", e i vari argomenti che iniziano con '-' sono dei flag che compiono operazioni aggiuntive.

Il processo corrente inizierà ad eseguire il comando `ls` (preso in questo caso dalle variabili di ambiente, ma solo perché è uno dei comandi di default di Linux) con i vari argomenti dell'array `args`. Il valore di terminazione per smettere di leggere argomenti è il valore speciale `NULL` all'interno dell'Array degli argomenti.

Avendo visto il funzionamento sia per `fork()` che `exec()`, vediamo un esempio di programma che li combina.

```
int main() {
  int pid = fork();
  if (pid == 0) {
    execvp("ls", {"\tmp", NULL})
  } else {
    printf("Processo padre.")
  }
}
```

Listing 4.3: Esempio di uso combinato di fork() e execvp(). Si ignorano possibili errori per brevità.

Avendo visto e compreso il funzionamento di questi due comandi, possiamo avere anche una visione più chiara di come funzioni un'istanza di terminale (shell), in quanto si basa su un loop di istruzioni molto preciso:

1. Far scrivere all'utente un comando.
2. Leggere il comando e dividerlo in token.
3. Creare un processo figlio che si occupi di eseguire il comando (`fork()`).
4. Far eseguire il comando al processo figlio (`execvp()`).
5. Attendere che il processo figlio termini l'esecuzione del comando (`wait()`).
6. Restituire il controllo al processo padre e ritornare all'inizio del loop.

Osservazione 4.7.2: Reale funzionamento di una shell

Una vera shell in realtà consente anche all'utente di eseguire nuovi comandi mentre altri stanno continuando in background. Questo può avvenire inserendo il carattere `&` alla fine di un comando, il che indicherà al sistema operativo di restituire il controllo all'utente subito dopo aver mandato in esecuzione il comando precedente.

4.8 Altre API di sistema

Di seguito un veloce elenco di altre API di sistema importanti ma non fondamentali come quelle descritte precedentemente.

exit(): si occupa di terminare un processo. E' presente in due varianti:

- `exit()`, che termina il processo in modo "lento" ma pulendo tutti i buffer e pulendo tutti gli stream.
- `_exit()`, che termina il processo in modo veloce ma senza pulire buffer e stream.

kill(): per inviare un segnale al processo figlio e "ucciderlo".

nice(): per aggiornare il livello di priorità di un processo. Tale livello è un numero intero che va da **-20** (**priorità massima**, solo il kernel) a **+19** (**priorità minima**, programmi utente).

pipe(): crea un canale di comunicazione tra due o più processi per permettere di condividere l'output di un processo con l'input di un altro.

5

Scheduling della CPU

Nei capitoli precedenti abbiamo accennato a come i sistemi moderni creino l'illusione di eseguire simultaneamente più processi "semplicemente" effettuando dei rapidissimi scambi di stato tra i processi in esecuzione.
In questo capitolo vedremo più nel dettaglio il ruolo dello **scheduler**, ovverosia quel modulo del kernel che si occupa di decidere, mediante un accorto **algoritmo di scheduling**, l'ordine dei processi da mandare in esecuzione.

L'obiettivo non sarà tuttavia fornire direttamente la soluzione adottata oggi dai sistemi moderni, bensì di guidare l'apprendimento imponendo dei vincoli e man mano rimuovendoli, per trovare standard sempre più efficaci.

5.1 Assunzioni sul carico di lavoro e metriche

Affinché lo scheduler possa organizzare i processi, bisogna definire il carico di lavoro e delle metriche secondo le quale stabilire questi confronti.

Definizione 5.1.1: Carico di lavoro

Il carico di lavoro descrive:

- Quanti processi stanno attualmente venendo eseguiti dal sistema.
- L'istante di tempo in cui viene creato un nuovo PCB.
- **Quanto tempo** è richiesto per ogni processo.
- **Quando** il processo dovrà fare operazioni di I/O.

Osservazione 5.1.1: Natura asincrona dei processi

E' doveroso fare un appunto sulla definizione soprastante.
Per lo scheduler è **impossibile** predirre gli ultimi due punti, in quanto i processi sono di per sè un'unità asincrona in cui alcune cose non sono deterimistiche.

Al fine di costruire il nostro percorso in maniera graduale verso un algoritmo di scheduling efficiente che funzioni per ogni sistema, dobbiamo partire letteralmente *dalle basi*, facendo delle assunzioni **molto comode** sulla natura del workload:

1. Ogni processo richiede la **stessa quantità di tempo** per essere eseguito.
2. Tutti i processi **arrivano allo stesso momento** ($t = 0$).
3. Prima di cambiare processo deve essere **completamente terminato il corrente** (nessuna interruzione del processo).
4. **Nessun** processo ha necessità di eseguire operazioni di **I/O**.
5. Il **tempo di esecuzione** di ogni processo è **noto a priori**.

Come vediamo sono delle assunzioni molto comode, che però piano piano andremo a calare per migliorare progressivamente l'algoritmo di scheduling che vogliamo progettare.

Oltre a queste assunzioni, che si permetteranno di definire l'algoritmo, dobbiamo anche avere dei criteri per permetterci di stabilire se un algoritmo performa meglio di un altro. Abbiamo principalmente due metriche:

Tempo di completamento (Turnaround Time): indica il tempo totale trascorso da quando il processo arriva al sistema a quando viene finito di eseguire.
La formula per il calcolo di questo parametro è la seguente:

$$T_{\text{turnaround}} = T_{\text{fine_processo}} - T_{\text{arrivo}}$$

Prendendo la media del tempo di completamento di tutti i processi schedulati, otteniamo l'importantissimo parametro di **Tempo di completamento medio**.

Tempo di risposta (Response Time): rappresenta il tempo da quando un processo arriva al sistema a quando arriva ad essere eseguito per la prima volta. Sostanzialmente indica quanto tempo un processo rimane in coda prima di essere considerato.
La formula per questo parametro è la seguente:

$$T_{\text{response}} = T_{\text{prima_esecuzione}} - T_{\text{arrivo}}$$

Questi appena descritti sono i parametri fondamentali nel calcolo dell'**equità** e delle **performance** di un algoritmo di scheduling.
In particolare:

Equità (Fairness): rappresenta in che misura il **tempo della CPU viene distribuito tra tutti i processi**. Un algoritmo è "**fair**" se garantisce a ogni processo di fare progressi regolari, evitando che qualche task rimanga bloccato ad aspettare per un tempo indefinito (starvation).

Prestazioni (Performance): rappresenta la **capacità di minimizzazione del turnaround time o del response time**.

Si può facilmente intuire come queste due filosofie siano contrapposte, pertanto si deve scegliere cosa si deve prediligere maggiormente in un algoritmo, oppure trovare l'equilibrio più vantaggioso tra le due.

5.2 I primi passi: FIFO e SJF

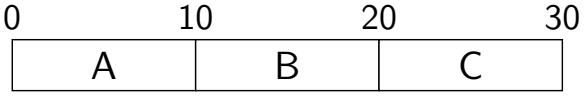
Avendo messo le basi per la nostra analisi degli algoritmi di scheduling, partiamo dalle basi, con due delle strategie più semplici.

5.2.1 FIFO - First In, First Out

E' forse la schedule più semplice a cui si potrebbe pensare: **i primi processi ad entrare sono anche i primi a venir eseguiti**.
Anche la sua implementazione risulta particolarmente semplice, in quanto consiste semplicemente nella gestione di una struttura dati *coda*. Vediamo un semplice esempio di questo algoritmo usando anche le nostre 5 assunzioni precedenti.

Supponiamo di avere tre job: **A**, **B** e **C** e che:

- Tutti arrivino allo stesso istante di tempo $t = 0$.
- Tutti abbiano la stessa durata di $10s$.



Andiamo adesso a calcolare il tempo di completamento per ogni processo e di conseguenza il tempo medio:

- A termina a $t = 10$. Tempo di completamento $= 10 - 0 = 10s$
- B termina a $t = 20$. Tempo di completamento $= 20 - 0 = 20s$
- C termina a $t = 30$. Tempo di completamento $= 30 - 0 = 30s$

Tempo di completamento medio: $\frac{10+20+30}{3} = 20s$.
Non è un brutto tempo, solo che stiamo lavorando con delle ipotesi molto comode.

Adesso, andiamo a rilassare l'Assunzione 1, quella sull'equa durata di tutti i processi.
Proviamo a ridisegnare lo schema precedente supponendo ora che $A = 100s, B = C = 10s$.



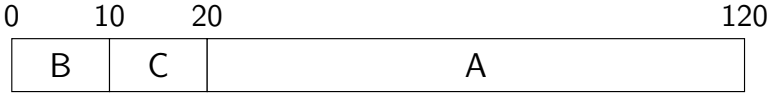
Se andassimo ora a calcolare il tempo di esecuzione minimo per ogni processo, il risultato sarebbe ben diverso:

- A termina a $t = 100$. Tempo di completamento $= 100 - 0 = 100s$
- B termina a $t = 110$. Tempo di completamento $= 110 - 0 = 110s$
- C termina a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$

Tempo di completamento medio: $\frac{100+110+120}{3} = 110s$.
Vediamo che la situazione è cambiata, e questo dato che i processi corti B e C sono bloccati dietro il processo lungo A e devono attenderne la fine. Nello studio degli algoritmi di scheduling, questo è detto **effetto convoglio**.

5.2.2 SJF - Shortest Job First

Per aggiustare questo effetto convoglio, evitando dunque che i procssi corti siano messi dopo uno più lungo di loro e rimangano bloccati, si attua la tecnica Shortest Job First (SJF) che, come suggerisce il nome, prevede di **eseguire per primo il processo più corto di tutti**.
Tenendo conto di questa nuova strategia, se andiamo a ridisegnare il diagramma precedente con $A = 100, B = C = 10$, vediamo come le cose cambino anche solo eseguendo per primi i processi corti.



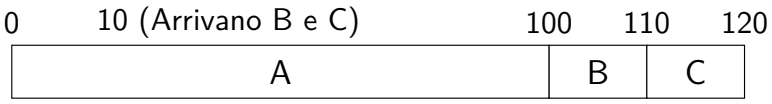
- B termina a $t = 10$. Tempo di completamento $= 10 - 0 = 10s$
- C termina a $t = 20$. Tempo di completamento $= 20 - 0 = 20s$
- A termina a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$

Facendo la media, otteniamo $\frac{10+20+120}{3} = 50s$ il che è un tempo inferiore per più della metà rispetto a prima!

Osservazione 5.2.1: Utilità del SJF

L'algoritmo SJF è **ottimale** per **minimizzare il tempo di completamento medio per processi che arrivano tutti allo stesso momento**.

Avendo lavorato sulla prima assunzione, riguardante la durata dei processi e avendo trovato un algoritmo che ci permette di rendere efficiente lo scheduling con quell'ipotesi rilassata, andiamo a rilassare anche a rimuovere la seconda assunzione, riguardante il fatto che **tutti i processi arrivano allo stesso tempo**.
Modifichiamo leggermente le variabili del nostro diagramma: $A = 100, t = 0; B = C = 10, t = 10$.



Analizzando ciò che accade:

1. Al tempo $t = 0$, l'unico processo presente è A e dunque lo scheduler deve necessariamente scegliere quello.
2. Al tempo $t = 10$ arrivano B e C, ma siccome il SJF non offre nessun meccanismo per gestire l'interruzione di un processo, questi dovranno attendere che A sia terminato prima di venir eseguiti.

Vediamo dunque che in questa casistica, avendo semplicemente rilassato la seconda ipotesi, **l'effetto convoglio è tornato**.

- A termina a $t = 100$. Tempo di completamento $= 100 - 0 = 100s$
- B termina a $t = 110$. Tempo di completamento $= 110 - 10 = 100s$
- C termina a $t = 120$. Tempo di completamento $= 120 - 10 = 110s$

Il tempo di completamento medio risulta essere $\frac{100+100+120}{3} = 103.3s$. Non ottimale.

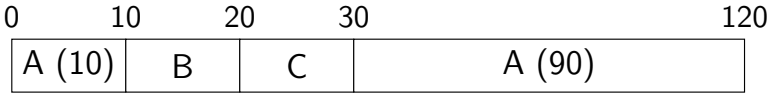
5.3 Interruzione dei processi e STCF

Per costruire un algoritmo di scheduling efficiente, dobbiamo rilassare la terza assunzione (i processi vanno avanti finché non vengono terminati).
Rilassare questa assunzione significa anche introdurre un meccanismo per memorizzare il *contesto* di un processo (ovvero i valori di tutti i registri utilizzati) per permettergli di ricominciare da dove si era interrotto. Come abbiamo già detto, però, questa abilità di interrompere è possibile grazie al Timer Interrupt, che è un dispositivo hardware.

Introduciamo così l'algoritmo **STCF - Shortest Time to Complete First** che opera nel seguente modo:

- Quando un processo entra nella coda dei processi, lo scheduler compara il tempo di esecuzione rimanente del processo corrente con quello totale del nuovo processo.
- Se il nuovo job ha un tempo di esecuzione inferiore rispetto al corrente, viene effettuato un cambio di contesto ed eseguito il nuovo processo fino a terminazione (o finché non ne arriva uno più corto).

Con questa nuova strategia, vediamo come si comporta la nostra schedule precedente: Andiamo ad illustrare nel dettaglio cosa accade:



1. Al tempo $t = 0$ è presente solo il processo A, dunque viene eseguito lui.
2. Al tempo $t = 10$ arrivano i processi B e C, entrambi della durata di $10s$.
    - Essendo arrivato un nuovo processo, viene comparato il tempo di esecuzione rimanente di $A(100 - 10 = 90s)$ con la durata del processo entrante $10s$.
    - Siccome il processo entrante è più corto, viene subito mandato in esecuzione.

Andiamo allora a vedere se è effettivamente migliorato qualcosa, a livello di tempi di completamento:

- A arriva a $t = 0$ e finisce a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$.
- B arriva a $t = 10$ e finisce a $t = 20$. Tempo di completamento $= 20 - 10 = 10s$.
- C arriva a $t = 10$ e finisce a $t = 30$. Tempo di completamento $= 30 - 10 = 20s$.

La media tra queste grandezze è $\frac{120+10+20}{3} = 50s$. Decisamente un miglioramento pensando ai $103.3s$ precedenti.

Osservazione 5.3.1: Utilità del STFC

L'algoritmo STCF è **ottimale** quando si devono **eseguire processi di cui si conosce la lunghezza** ma che **possono arrivare in momenti diversi**.

Tuttavia, anche questo algoritmo non è sicuramente esente da difetti. Immaginiamo una situazione in cui il processo A di durata $100s$ inizia ad essere eseguito a tempo $t = 0$. Supponiamo che poi ogni 10 secondi arrivi un processo di durata inferiore ad A: verrebbe data priorità a lui, in quanto più corto. A lungo andare verrebbero eseguiti sempre processi di breve durata, ma quello più lungo rimarrebbe in coda ad attendere. Tale situazione è detta **Starvation**, che è un problema di **equità**.

5.4 Round Robin (RR)

Supponiamo di star eseguendo un processo su un sistema multiutente. L'utente A sta modificando un file di testo e l'utente B sta compilando un programma.
Supponendo che la compilazione richieda $10s$, gli algoritmi SJB e STCF darebbero la precedenza a questo processo. Ciò vuol dire però che all'utente A in quei $10s$ verranno ignorate le pressioni di tasti. Ci rendiamo conto di come, in una situazione reale, tali algoritmi non siano molto efficienti.

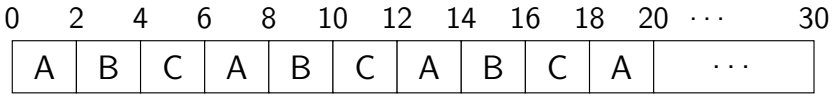
Ecco perché introduciamo l'algoritmo **Round Robin (RR)**, che si occupa di schedulare i processi seguendo un loop circolare:

- Anziché eseguire un processo fino alla fine, lo esegue per un breve lasso di tempo.
- Alla fine di questo lasso di tempo, il processo viene spostato alla fine della coda.
- Viene eseguito il processo attualmente in testa, e così via finché i processi non sono esauriti...

Il lasso di tempo entro il cui un processo viene eseguito è detto **Time Slice** o **Scheduling Quantum**, il quale deve rispettare le seguenti caratteristiche:

- Deve essere abbastanza lungo da ammortizzare i costi del context switch.
- Deve essere abbastanza corto da creare l'illusione dell'esecuzione simultanea.

Vediamo un esempio pratico, come fatto prima per i precedenti algoritmi: supponiamo di avere i processi A, B, C, tutti che arrivano a $t = 0$ e dalla durata di $10s$ ciascuno; il time-slice è impostato a $2s$. Stavolta andiamo a calcolare i **tempi di risposta** (che ricordiamo



essere la differenza tra quando un processo arriva in coda e quando viene eseguito per la prima volta).

- A arriva a $t = 0$ e viene eseguito per la prima volta a $t = 0$. Tempo di risposta $0 - 0 = 0s$.
- B arriva a $t = 0$ e viene eseguito per la prima volta a $t = 2$. Tempo di risposta $2 - 0 = 2s$.
- C arriva a $t = 0$ e viene eseguito per la prima volta a $t = 4$. Tempo di risposta $4 - 0 = 4s$.

Otteniamo come il tempo medio di risposta sia $\frac{0+2+4}{3} = 2s$, il che è un tempo di attesa **significativamente basso**. Il lettore può provare da solo a calcolare tale grandezza in riferimento ad un algoritmo come SJF.

Osservazione 5.4.1: Utilità del Round Robin

Come abbiamo visto, l'algoritmo Round Robin è **ottimale** per **minimizzare i tempi di risposta** (cioè per garantire che i processi vengano tutti "serviti" in un tempo ragionevole), tuttavia risulta terribile per quanto riguarda il tempo di completamento.

Considerando sempre l'esempio precedente, abbiamo che A termina a $t = 26$, B termina a $t = 28$, C termina a $t = 30$. Vediamo come il tempo di completamento minimo sia di $\frac{26+28+30}{3} = 28s$, il che è un tempo anche peggiore degli algoritmi SFJ e STCF.

Possiamo dunque fare una considerazione generale per quanto riguarda i tempi di risposta e di completamento.

Osservazione 5.4.2: Trade-off tra risposta e completamento

- Ogni politica che **ottimizza il tempo di risposta avrà un downgrade sul tempo di completamento**.

- Ogni politica che **ottimizza il tempo di completamento avrà un downgrade sul tempo di risposta**.

5.5 Incorporazione dell'I/O

Purtroppo, è arrivato anche il momento di rilassare la quarta assunzione, riguardante l'assenza di operazioni di I/O da completare, anche perché queste sono molto più frequenti di quanto ci si possa immaginare.
Come sappiamo avendo studiato gli stati di un processo, quando il job deve svolgere un'operazione di I/O si deve:

1. Mettere il processo in stato `BLOCKED`.
2. Attendere che l'operazione sia completata, e intanto eseguire altri processi.
3. Una volta completata l'operazione, il processo viene rimesso nella coda dei processi `READY`.

Il punto di un buono scheduler è proprio garantire che venga eseguito un altro processo nel frattempo che il corrente è impegnato in un'operazione di I/O.
La sezione di tempo in cui un processo viene eseguito è detta **CPU Burst**, mentre quella in cui il processo attende l'I/O si chiama **I/O Wait**.

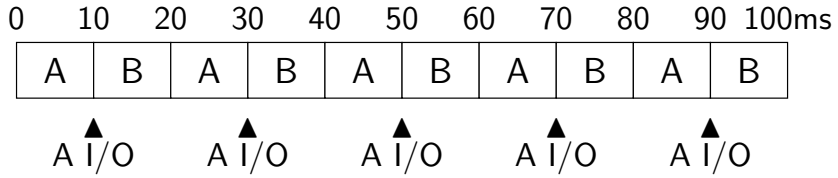
Tutto bello ma...come lo implementiamo nei nostri algoritmi di scheduling?
Il trucco è trattare ogni CPU Burst come un sotto-processo indipendente, che viene eseguito e schedulato per conto suo.

- Se il processo A ha bisogno di un totale di 50ms di CPU, dividiamo questo tempo in 5 CPU Burst ognuno da 10ms, separati ognuno da operazioni di I/O.
- Ogni CPU Burst da 10ms viene schedulato come se fosse un processo indipendente.

Vediamo un esempio di questa implementazione con l'algoritmo STCF, con il seguente esempio:

- Il processo A ha 10ms di CPU Burst e 10ms di I/O wait (il tutto ripetuto per 5 volte).
- Il processo B ha 50ms di solo CPU Burst.

Ricordiamo che l'algoritmo di scheduling è il STCF, dunque il workflow è il seguente:



1. Il processo A viene schedulato per primo perché la sua durata (10ms, ricordiamo che A è stato suddiviso in sotto-processi indipendenti) è inferiore a quella del processo B (50ms).

2. Dopo 10ms, il processo A entra in I/O Wait per 10ms, tempo durante il quale verrà eseguito il processo B.

3. Avanti così finché uno dei due processi non termina...